Lecture notes of finite element methods

Instructor: Jie Liu

Based on: **Claes Johnson, "Numerical solution of partial differential equations by the finite element method "** Cambridge University Press (1987)

Other related finite element books and free finite element packages:

- S. C. Brenner and L. R. Scott, "The mathematical theory of finite element methods", Springer-Verlag, (1994)
- P. G. Ciarlet, "The finite element method for elliptic problems", North-Holland (1987)
- H. C. Elman, D. J. Silvester and A. J. Wathen, "Finite Elements and Fast Iterative Solvers", Oxford University Press, (2005) (with MATLAB code)
- M. S. Gockenbach, "Understanding and implementing the finite element method", SIAM, (2006) (with MATLAB code)
- A. Schmidt and K. G. Siebert, "Design of adaptive finite element software: The finite element toolbox ALBERTA", Springer LNCSE Series 42 (2005). See also `http://www.alberta-fem.de/` (with C code)

Contents

1 Elliptic problem in 1d **4**
- 1.1 finite element method and error estimates 4
- 1.2 extra reading: finite difference method and error estimates . . 5

2 The Hilbert space $L^2(\Omega)$, $H^m(\Omega)$ and $H_0^m(\Omega)$ **7**

3 Elliptic problem in 2d **8**
- 3.1 Assembly of global stiffness matrix 9
- 3.2 Neumann boundary condition 10

4 Abstract formulation of the FEM for elliptic problems **11**
- 4.1 discretization and an error estimate 12
- 4.2 examples 13

5 Some finite element space **15**

6 Approximation theory **20**
- 6.1 L^∞ interpolation error estimate with piecewise linear functions in 2d 20
- 6.2 L^2 and H^1 interpolation error estimates 22
- 6.3 examples 25

7 Error estimates for elliptic problem **27**
- 7.1 regularity of exact solution 27
- 7.2 an error estimate in the $L^2(\Omega)$ norm 28
- 7.3 inverse inequality and the condition number of the stiffness matrix 28

8 Parabolic problem **31**
- 8.1 semi-discretization in space 31
- 8.2 backward Euler 35
- 8.3 Crank-Nicolson 36
- 8.4 discontinuous Galerkin 36
  - 8.4.1 unconditional stable when $f = 0$ 37
  - 8.4.2 stability of discrete time derivative for the $q = 0$ and $f = 0$ case 38
  - 8.4.3 error estimate for the $q = 0$ case 39

9 Mixed finite element method and fluid dynamics **42**
9.1 derivation of the equations 42
9.2 PDE theory in divergence free space 43
9.3 another weak form and inf-sup condition 44
9.4 error estimates 45

10 ALBERTA **49**
10.1 sample ALBERTA files 51
10.2 example of solving Poisson equation without adaptation 58

11 Homework and Projects **65**
11.1 Homework 65
11.2 Project 65

12 Extra reading: solving parabolic equation with finite difference **66**
12.1 method of line 66
12.2 local truncation error 67
12.3 von Neumann analysis: stability and accuracy 67
12.4 energy method 69

1 Elliptic problem in 1d

consider

$$-u''(x) = f(x) \text{ on } [0,1], \qquad u(0) = u(1) = 0. \tag{1.1}$$

1.1 finite element method and error estimates

Let $V = \{v : v \in H_0^1([0,1])\}$ and

$$F(v) = \frac{1}{2}(v', v') - (f, v).$$

Then u satisfies the following two conditions

$$F(u) \leq F(v), \qquad \forall v \in V \tag{1.2}$$

$$(u', v') = (f, v), \qquad \forall v \in V. \tag{1.3}$$

One can show (1.1) $\Rightarrow$ (1.2) $\Leftrightarrow$ (1.3). Instead of (1.1), (1.2) or (1.3) is going to be our starting point of FEM.

Now, consider the mesh $\mathcal{T}_h : 0 = x_0 < x_1 < ... < x_{M+1} = 1$ on [0,1] and let's introduce the finite dimensional space

$$V_h = \{v : v \in H_0^1([0,1]), v \text{ is piecewise linear on mesh } \mathcal{T}_h\}$$

and the basis function $\{\varphi_j, j = 1, ..., M\}$

$$\varphi_j(x) \in V_h, \quad \varphi_j(x_i) = 1 \text{ when } i = j, \quad \varphi_j(x_i) = 0 \text{ when } i \neq j.$$

Then any function $v \in V_h$ has the representation $v(x) = \sum_{i=1}^{M} \eta_i \varphi_i(x)$.

The FEM is find $u_h \in V_h$ such that

$$F(u_h) \leq F(v_h), \qquad \forall v_h \in V_h$$

or we can further write it

$$(u_h', v_h') = (f, v_h), \qquad \forall v_h \in V_h. \tag{1.4}$$

FEM can be summarized in the following sentence: Restrict the weak form of the differential equation onto a finite-dimensional function space.

Let

$$u_h(x) = \sum_{i=1}^{M} \xi_i \varphi_i(x).$$

Then (1.4) leads to $A\xi = b$ where $\xi = (\xi_1, ..., \xi_M)^\tau$, $b = (b_1, ..., b_M)^\tau$, $A = (a_{ij})$

$$b_i = (f, \varphi_i), \qquad a_{i,j} = (\varphi_i', \varphi_j').$$

Let $h_j = x_j - x_{j-1}$, one can compute and obtain

$$(\varphi_j', \varphi_j') = \frac{1}{h_j} + \frac{1}{h_{j+1}} \text{ for } j = 1, ..., M$$

and

$$(\varphi_j', \varphi_{j-1}') = -\frac{1}{h_j} \text{ for } j = 2, ..., M.$$

When the mesh is uniform with $h_j = h$, $A = \frac{1}{h}\text{tridiag}(-1, 2, -1)$. Up to an factor h, one will obtain the same matrix if solving (1.1) by standard finite difference method.

The error satisfies the equation

$$((u - u_h)', v_h') = 0, \qquad \forall v_h \in V_h \tag{1.5}$$

and hence $((u - u_h)', (u - u_h') = ((u - u_h)', (u - v_h)')$ for any $v_h \in V_h$ which by Cauchy-Schwarz implies

$$\|(u - u_h)'\| \le \|(u - v_h)'\|, \qquad \forall v_h \in V_h. \tag{1.6}$$

Let $\tilde{u}_h \in V_h$ be the interpolant of u. For any x, suppose $x \in [x_i, x_{i+1}]$ and let $h = \max_k(x_{k+1} - x_k)$, then

$$\begin{aligned} |u'(x) - \tilde{u}_h'(x)| &= \left| u'(x) - \frac{u(x_{i+1}) - u(x_i)}{x_{i+1} - x_i} \right| = \left| \frac{1}{x_{i+1} - x_i} \int_{x_i}^{x_{i+1}} u'(x) - u'(s) ds \right| \\ &\le h \max_{y \in [0,1]} |u''(y)|. \end{aligned}$$

So we obtain

$$\|(u - u_h)'\| \le h \max_{y \in [0,1]} |u''(y)|. \tag{1.7}$$

1.2 extra reading: finite difference method and error estimates

To use finite difference to solve (1.1), we will use

$$-\frac{u_{j+1} - 2u_j + u_{j-1}}{h^2} = f_j, \qquad u_0 = u_N = 0. \tag{1.8}$$

By introducing $D_+u_j = (u_{j+1} - u_j)/h$ and $D_-u_j = (u_j - u_{j-1})/h$, (1.8) can be written as $-D_+D_-u_j = f_j$. The exact solution satisfies $-D_+D_-u(t_j) = -u''(t_j) + g_jh^2 = f_j + g_jh^2$. So the error equation is $-D_+D_-e_j = g_jh^2$.

Note that for any k, assuming $v_0 = 0$ and using triangle inequality

$$|v_k| \leq \sum_j |v_{j+1} - v_j| \frac{1}{\sqrt{h}} \sqrt{h} \leq \sqrt{\sum_j |v_{j+1} - v_j|^2 \frac{1}{h}} \sqrt{\sum_j h} = \|D_+v\|$$

where $\|D_+v\|^2 = \sum_j |D_+v_j|^2 h$. So, from the error equation $-D_+D_-e_j = g_jh^2$, multiply it by e_j, times h, and then summation on j, we get

$$\|D_+e\|^2 \leq \sum_j g_jh^3|e_j| \leq Ch^2 \max_j |e_j| \leq Ch^2\|D_+e\|$$

where we have used the summation by part $-h\sum_j e_j(D_+D_-e_j) = \|D_+e\|^2$. This proves

$$\max_j |e_j| \leq \|D_+e\| \leq Ch^2.$$

So, it's 2nd order accurate for central difference scheme.

2 The Hilbert space $L^2(\Omega)$, $H^m(\Omega)$ and $H_0^m(\Omega)$

Let $\Omega \subset \mathbb{R}^n$. $L^2(\Omega)$ consists of functions on Ω that are square integrable.

$H^m(\Omega)$ consists of those functions $v \in L^2(\Omega)$ for which all partial derivatives $D^\alpha v$ (in the distribution sense) with $|\alpha| \le m$, are in $L^2(\Omega)$, i.e., for each α with $|\alpha| \le m$, there is a function $D^\alpha v \in L^2(\Omega)$ so that

$$\int_\Omega v D^\alpha \varphi dx = (-1)^{|\alpha|} \int_\Omega D^\alpha v \varphi dx, \qquad \forall\, \varphi \in C_0^\infty(\Omega).$$

Here $\alpha = (\alpha_1, ..., \alpha_n)$ is a multi-index and $|\alpha| = \alpha_1 + ... + \alpha_n$.

$H^m(\Omega)$ is equipped with the norm

$$\|v\|_{H^m} = \left(\sum_{|\alpha| \le m} \int_\Omega |D^\alpha v|^2 dx \right)^{1/2}. \tag{2.1}$$

We will also need to use the semi-norm

$$|v|_{H^m} = \left(\sum_{|\alpha| = m} \int_\Omega |D^\alpha v|^2 dx \right)^{1/2}. \tag{2.2}$$

$H_0^m(\Omega)$ consists of functions in $H^m(\Omega)$ such that the function and its α-th derivatives are zero on $\Gamma = \partial\Omega$ for any $|\alpha| \le m-1$. In particular,

$$H_0^1(\Omega) = \{v \in H^1 : v = 0 \text{ on } \Gamma\}. \tag{2.3}$$

$$H_0^2(\Omega) = \{v \in H^1 : v = \partial_n v = 0 \text{ on } \Gamma\} \tag{2.4}$$

since the tangential derivative $\partial_\tau v = 0$ already follows from $v = 0$ on Γ. Indeed, we can define $H_0^m(\Omega)$ as the closure of $C_0^\infty(\Omega)$ under the norm $H^m(\Omega)$. Then the boundary condition follows from the trace theorem

$$\|v\|_{H^{m-1/2}(\Gamma)} \le C\|v\|_{H^m(\Omega)}. \tag{2.5}$$

Why? consider $C_0^\infty(\Omega) v_i \to v$, $\|v\|_{H^{m-1/2}(\Gamma)} = \|v - v_i\|_{H^{m-1/2}(\Gamma)} \le C\|v - v_i\|_{H^m(\Omega)} \to 0$.

Besides this trace theorem, we also need the following *compact* imbedding results

$$H^m(\Omega) \subset C^{[m-\frac{n}{2}]}(\Omega), \qquad \text{if } m - \frac{n}{2} > 0 \tag{2.6}$$

and

$$H^m(\Omega) \subset L^q(\Omega) \text{ for any } q \in [1, \infty), \qquad \text{if } m - \frac{n}{2} = 0. \tag{2.7}$$

3 Elliptic problem in 2d

consider

$$-\Delta u = f \text{ in } \Omega, \qquad u = 0 \text{ on } \Gamma. \tag{3.1}$$

Let $V = \{v : v \in H_0^1(\Omega)\}$ and we have a weak formulation of (3.1):

$$a(u, v) = (f, v), \qquad \forall v \in V. \tag{3.2}$$

where $a(u, v) = \int_\Omega \nabla u \cdot \nabla v = \int_\Omega (u_{x_1} v_{x_1} + u_{x_2} v_{x_2}) dx$.

Now, consider the mesh $\mathcal{T}_h = \{K_1, ..., K_m\}$ which form a triangulation of Ω. We require that *no vertex of one triangle lies on the edge of another triangle.* Let's introduce the finite dimensional space

$$V_h = \{v : v \in H_0^1(\Omega), v|_K \text{ is linear} \, \forall K \in \mathcal{T}_h\}.$$

Suppose the nodes are $\{N_i, i = 1, ..., M\}$, then the basis function are $\{\varphi_j, j = 1, ..., M\}$ with

$$\varphi_j(x) \in V_h, \quad \varphi_j(N_i) = 1 \text{ when } i = j, \quad \varphi_j(N_i) = 0 \text{ when } i \neq j.$$

Then, any function $v \in V_h$ has the representation

$$v(x) = \sum_{i=1}^{M} \eta_i \varphi_i(x).$$

The FEM is find $u_h \in V_h$ such that

$$a(u_h, v_h) = (f, v_h), \qquad \forall v_h \in V_h. \tag{3.3}$$

Let

$$u_h(x) = \sum_{i=1}^{M} \xi_i \varphi_i(x).$$

Then (3.3) leads to

$$A\xi = b \tag{3.4}$$

where $\xi = (\xi_1, ..., \xi_M)^\tau$, $b = (b_1, ..., b_M)^\tau$, $A = (a_{ij})$

$$b_i = (f, \varphi_i), \qquad a_{i,j} = a(\varphi_i, \varphi_j).$$

Define $a_K(\varphi_i, \varphi_j) = \int_K \nabla \varphi_i \cdot \nabla \varphi_j dx$. Then $a(\varphi_i, \varphi_j) = \sum_{K \in \mathcal{T}_h} a_K(\varphi_i, \varphi_j)$.

Let N_i, N_j and N_k be the vertices of the triangle K. We call the 3×3 matrix

$$\begin{pmatrix} a_K(\varphi_i, \varphi_i), & a_K(\varphi_i, \varphi_j), & a_K(\varphi_i, \varphi_k) \\ a_K(\varphi_j, \varphi_i), & a_K(\varphi_j, \varphi_j), & a_K(\varphi_j, \varphi_k) \\ a_K(\varphi_k, \varphi_i), & a_K(\varphi_k, \varphi_j), & a_K(\varphi_k, \varphi_k) \end{pmatrix}$$

the element stiffness matrix for K. For the triangle K with vertices at $(0, 0)$, $(h, 0)$ and $(0, h)$, the element stiffness matrix is given by

$$\begin{pmatrix} 1, & -1/2, & -1/2 \\ -1/2, & 1/2, & 0 \\ -1/2, & 0, & 1/2 \end{pmatrix}.$$

Let Ω is a square with side length 1 and let $\mathcal{T}_h$ be the uniform triangulation of Ω with lexicographic ordering of the nodes, then the matrix A in (3.4) reads the same as the finite difference matrix of $-\Delta u$.

The error satisfies the equation

$$a(u - u_h, v_h) = 0, \qquad \forall v_h \in V_h \tag{3.5}$$

and hence $a(u - u_h, u - u_h) = a(u - u_h, u - v_h)$ for any $v_h \in V_h$ which by Cauchy-Schwarz implies

$$\|\nabla(u - u_h)\| \leq \|\nabla(u - v_h)\|, \qquad \forall v_h \in V_h. \tag{3.6}$$

Let $\tilde{u}_h \in V_h$ be the interpolant of u, we will prove later that

$$\|\nabla(u - \tilde{u}_h)| \leq Ch. \tag{3.7}$$

where C depends on the size of the 2nd derivative of u and the smallest angle of the triangles $K \in \mathcal{T}_h$. Therefore $\|\nabla(u - u_h)\| \leq Ch$.

We will also prove later that

$$\|u - u_h\| \leq Ch^2. \tag{3.8}$$

3.1 Assembly of global stiffness matrix

Suppose the triangles are ordered from 1 to N. Construct a $3 \times N$ matrix T with $T(j, n)$ $(j = 1, 2, 3)$ being the global numbering of vertices of triangle K_n.

Assembly of global stiffness matrix

```
Set A(i,j) = 0 for i,j = 1,...,M (with M = # of nodes)
for n = 1,...,N (with N = # of triangles)
    for i = 1,..,3
        for j = 1,..,3
            A(T(i,n),T(j,n)) = A(T(i,n),T(j,n)) + a_{K_n}(φ_i,φ_j)
        end
    end
end
```

3.2 Neumann boundary condition

Consider

$$-\Delta u + u = f \text{ in } \Omega, \qquad \partial_n u = g \text{ on } \Gamma. \tag{3.9}$$

Recall

$$-\int_\Omega \Delta u v = \int_\Omega \nabla u \nabla v - \int_\Gamma (\partial_n u) v.$$

Let $V = \{v : v \in H^1(\Omega)\}$ and we have a variation formulation, or called weak formulation of (3.9):

$$a(u,v) = (f,v) + (g,v)_\Gamma, \qquad \forall v \in V. \tag{3.10}$$

where $a(u,v) = \int_\Omega \nabla u \cdot \nabla v + uv$. $(g,v)_\Gamma = \int_\Gamma gvds$.

Let

$$V_h = \{v : v \in H^1(\Omega), v|_K \text{ is linear} \forall K \in \mathcal{T}_h\}.$$

Then the finite element formulation is

$$a(u_h, v_h) = (f, v_h) + (g, v_h)_\Gamma, \qquad \forall v_h \in V_h. \tag{3.11}$$

For this problem, we also have the following error estimates

$$\|u - u_h\|_{H^1(\Omega)} \leq \inf_{v_h \in V_h} \|u - v_h\|_{H^1(\Omega)} \leq Ch.$$

Dirichlet boundary condition are built into the finite element space while Neumann boundary condition as well as Robin boundary condition are built into the weak formulation.

4 Abstract formulation of the FEM for elliptic problems

Let V be a Hilbert space with scalar product $(\cdot,\cdot)_V$ and norm $\|\cdot\|_V$. Suppose $a(\cdot,\cdot)$ is a bilinear form on $V \times V$ such that

(1) $a(\cdot,\cdot)$ is symmetric

(2) $a(\cdot,\cdot)$ is continuous, namely there is a constant γ so that for any $v, w \in V$, $|a(v,w)| \leq \gamma \|v\|_V \|w\|_V$.

(3) $a(\cdot,\cdot)$ is coercive, namely there is a constant $\alpha > 0$ so that for any $v, \in V$, $|a(v,v)| \geq \alpha \|v\|_V^2$.

Suppose L a bounded linear functional on V, i.e. there is a constant $\Lambda > 0$ so that for any $v, \in V$

$$|L(v)| \leq \Lambda \|v\|_V.$$

Let us now consider the problem: Find $u \in V$ such that

$$F(u) = \min_{v \in V} F(v), \qquad \text{where } F(v) = \frac{1}{2} a(v,v) - L(v) \tag{4.1}$$

and another problem: Find $u \in V$ such that

$$a(u,v) = L(v), \qquad \forall\, v \in V. \tag{4.2}$$

Theorem 4.1 *The problem (4.1) and (4.2) are equivalent, i.e., $u \in V$ satisfies (4.1) if and only if u satisfies (4.2). Moreover, there exists a unique solution $u \in V$ of these problems and the following stability estimates holds:*

$$\|u\|_V \leq \frac{\Lambda}{\alpha}. \tag{4.3}$$

Remark: Even without the symmetry condition (1), one can still prove the existence and uniqueness of $u \in V$ such that

$$a(u,v) = L(v), \qquad \forall v \in V$$

and the stability estimate (4.3). However, in this case there is no associated minimization problem.

4.1 discretization and an error estimate

Let V_h be a finite dimensional subspace of V of dimension M. Let $\{\varphi_1, ..., \varphi_M\}$ be a basis for V_h. Then any $v \in V_h$ has the representation $v = \sum_{i=1}^{M} \eta_i \varphi_i$.

We can now formulate the discrete analogues of the problem (4.1) and (4.2): Find $u_h \in V_h$ such that

$$F(u_h) = \min_{v_h \in V_h} F(v_h), \qquad \text{where } F(v_h) = \frac{1}{2} a(v_h, v_h) - L(v_h) \tag{4.4}$$

which is equivalent to: Find $u_h \in V_h$ such that

$$a(u_h, v_h) = L(v_h), \qquad \forall\, v_h \in V_h. \tag{4.5}$$

By linearity in test function, (4.5) is equivalent to

$$a(u_h, \varphi_j) = L(\varphi_j), \qquad \forall\, j = 1, ..., M. \tag{4.6}$$

Let $u_h = \sum_{i=1}^{M} \xi_i \varphi_i$, and

$$A = (a_{ij}) = (a(\varphi_i, \varphi_j)) \in \mathbb{R}^M \times \mathbb{R}^M, \qquad b = (b_i) = (L(\varphi_i)) \in \mathbb{R}^M.$$

Then (4.6) can be written as

$$A\xi = b. \tag{4.7}$$

Similarly, (4.4) can be written as

$$\frac{1}{2}\xi \cdot A\xi - b \cdot \xi = \min_{\eta \in \mathbb{R}^M} \left(\frac{1}{2}\eta \cdot A\eta - b \cdot \eta \right). \tag{4.8}$$

We also know A is symmetric and

$$\eta \cdot A\eta = a(v, v) \geq \alpha \|v\|^2 > 0.$$

Theorem 4.2 *(existence, uniqueness, stability) There exists a unique solution $\xi \in \mathbb{R}^M$ to the equivalent problem (4.7) and (4.8). Furthermore, $u = \sum_{i=1}^{M} \xi_i \varphi_i$ satisfies*

$$\|u_h\|_V \leq \frac{\Lambda}{\alpha}.$$

Theorem 4.3 *(error estimate)*

$$\|u - u_h\|_V \leq \frac{\gamma}{\alpha}\|u - v_h\|_V \qquad \forall\, v_h \in V_h.$$

If we define energy norm $\|v\|_a = a(v, v)$ *(which is equivalent to the norm* $\|\cdot\|_V$*), then*

$$\|u - u_h\|_a \leq \|u - v_h\|_a \qquad \forall\, v_h \in V_h.$$

In another word, u_h *is the best approximation of* u *in* V_h *with respect to the energy norm. This consistent with the fact that* u_h *is the projection of* u *onto* V_h *with respect to the scalar product* $a(\cdot, \cdot)$*.*

4.2 examples

Example 1 (Dirichlet problem: $-\Delta u = f$ in $\Omega \subset \mathbb{R}^2$, $\quad u = 0$ on Γ)
Let $V = H_0^1(\Omega)$,

$$a(v, w) = \int_\Omega (\nabla v \cdot \nabla w) dx, \qquad L(v) = \int_\Omega f v dx.$$

Example 2 (Neumann problem: $-\Delta u + u = f$ in $\Omega \subset \mathbb{R}^2$, $\quad \partial_n u = g$ on Γ)
Let $V = H^1(\Omega)$,

$$a(v, w) = \int_\Omega (\nabla v \cdot \nabla w + vw) dx, \qquad L(v) = \int_\Omega f v dx + \int_\Gamma g v ds.$$

Example 3 (Biharmonic in 1d: $\frac{d^4 u}{dx^4} = f$ on $[0, 1]$, $u(0) = u(1) = u'(0) = u'(1) = 0$)
Let $V = H_0^2([0, 1])$

$$a(v, w) = \int_0^1 v'' w'' dx, \qquad L(v) = \int_0^1 f v dx.$$

Example 4 (Biharmonic in 2d: $\Delta\Delta u = f$ in $\Omega \subset \mathbb{R}^2$, $u = \partial_n u = 0$ on Γ)
Let $V = H_0^2(\Omega)$,

$$a(v, w) = \int_\Omega \Delta v \Delta w dx, \qquad L(v) = \int_\Omega f v dx.$$

Example 5 (Convection-diffusion equation in 2d: $-\mu\Delta u+\beta_1 u_{x_1}+\beta_2 u_{x_2}+u = f$ in $\Omega \subset \mathbb{R}^2$, $u = 0$ on Γ)
Let $V = H_0^1(\Omega)$,

$$a(v, w) = \int_\Omega (\mu\nabla v \cdot \nabla w + (\beta_1 v_{x_1} + \beta_2 v_{x_2} + v)w)dx, \qquad L(v) = \int_\Omega fvdx.$$

Note that $a(\cdot, \cdot)$ (hence A) is no longer symmetric.

Example 6 (Variable coefficient) Consider the problem

$$-q_i = k_i(x)u_{x_i}, \qquad \nabla \cdot q = f \qquad \text{in } \Omega \subset \mathbb{R}^3$$

with boundary condition

$$u = 0 \text{ on } \Gamma_1 \qquad -q \cdot n = g \text{ on } \Gamma_2.$$

Let $V = \{v \in H^1(\Omega), v = 0 \text{ on } \Gamma_1\}$,

$$a(v, w) = \int_\Omega \sum_{i=1}^{3} k_i(x)v_{x_i}w_{x_i}dx, \qquad L(v) = \int_\Omega fvdx + \int_{\Gamma_2} gvds.$$

5 Some finite element space

We shall present some finite element spaces V_h consist of piecewise polynomials on triangulation $\mathcal{T}_h = \{K_1, ..., K_N\}$ of a bounded domain Ω.

Since V_h consist of piecewise polynomials,

$$V_h \subset H^1(\Omega) \Longleftrightarrow V_h \subset C^0(\bar{\Omega})$$

$$V_h \subset H^2(\Omega) \Longleftrightarrow V_h \subset C^1(\bar{\Omega}).$$

To define a finite element space V_h, we will have to specify

(1) the triangulation $\mathcal{T}_h = \{K\}$ of the domain

(2) the nature of the functions v in V_h on each element K (e.g. linear, quadratic, cubic, etc.).

(3) the parameters to be used to describe the functions in V_h. Those parameters are called *global degrees of freedom (dof)*. (e.g. we can use the values at the node points of $\mathcal{T}_h$, namely $v(x) = \sum_{i=1}^{M} v(N_i)\varphi_i(x)$ with $\{N_1, ..., N_M\}$ the nodes and $\{v(N_i)\}$ is the global dof.)

Let

$$\begin{aligned} P_r(K) &= \{v : v \text{ is a polynomial of degree} \leq r \text{ on } K\} \\ &= \{v : v(x) = \sum_{0 \leq i+j \leq r} b_{ij} x_1^i x_2^j\} \quad \text{when } K \subset \mathbb{R}^2. \end{aligned}$$

We have

$$\dim P_r(K) = \frac{(r+1)(r+2)}{2} \quad \text{when } K \subset \mathbb{R}^2.$$

Example 1 (P_1 Lagrange element on $\mathbb{R}^2$)

Theorem 5.1 *Let $K \in \mathcal{T}_h$ be a triangles with vertices a^i $(i = 1, 2, 3)$ A function $v \in P_1(K)$ is uniquely determined by the degree of freedom: $v(a^i)$ $(i = 1, 2, 3)$.*

Proof: We only need to prove that $v(a^i) = 0$ implies $v(x) = 0$.

Take any boundary of K, call it S_1. Suppose the equation of S_1 on $\{x_1, x_2\}$ plan is $\lambda_1(x_1, x_2) = 0$. (e.g. $\lambda_1(x_1, x_2) = x_1 + x_2 - h = 0$). Because $v(x)$ vanishes on S_1, using long division of polynomials, we can prove that

$\lambda_1(x)$ is a factor of $v(x)$. So $v(x) = \gamma\lambda_1(x)$ and γ has to be a number because both $v(x)$ and $\lambda_1(x)$ are first order polynomials. Then we evaluate $v(x)$ at a^1 which is the vertex opposite to S_1. We conclude $\gamma = 0$. $\square$

Define the finite element space

$$V_h = \{v \in C^0(\bar{\Omega}) : v|_K \in P_1(K),\ \forall\, K \in \mathcal{T}_h\}.$$

Now, let $v_i \in P_1(K_i)$, $i = 1, 2$, where K_1 and K_2 are two triangles with common side S. If v_1 and v_2 take the same values at the two end points of S, then $v_1 = v_2$ on S. This means

$$V_h = \{v : v|_K \in P_1(K),\ \forall\, K \in \mathcal{T}_h, v \text{ is continuous at the nodes}\}.$$

Example 2 (P_2 Lagrange element on $\mathbb{R}^2$)

Theorem 5.2 *A function $v \in P_2(K)$ is uniquely determined by the degree of freedom:*

$$\begin{aligned} &v(a^i) \qquad i = 1, 2, 3 \\ &v(a^{ij}) \qquad i < j, \quad i, j = 1, 2, 3. \end{aligned}$$

where $a^{ij} = \frac{1}{2}(a_i + a_j)$. Define the finite element space

$$V_h = \{v \in C^0(\bar{\Omega}) : v|_K \in P_2(K),\ \forall\, K \in \mathcal{T}_h\}.$$

One can prove like before that

$$V_h = \{v : v|_K \in P_2(K),\ \forall\, K \in \mathcal{T}_h,\ v \text{ is continuous at the nodes and the middle points}\}.$$

Example 3 (P_3 Lagrange element on $\mathbb{R}^2$)

Theorem 5.3 *A function $v \in P_3(K)$ is uniquely determined by the degree of freedom:*

$$\begin{aligned} &v(a^i),\, v(a^{iij}) \qquad i, j = 1, 2, 3, \quad i \neq j \\ &v(a^{123}) \end{aligned}$$

Proof: We only need to prove that $v(a^i) = v(a^{iij}) = v(a^{123}) = 0$ implies $v(x) = 0$.

Because $v(x)$ vanishes at the 3 boundaries of K, using long division of polynomials, we can prove that $\lambda_i(x)$ is a factor of $v(x)$ for $i = 1, 2, 3$, where λ_i are the equation for the i-th boundary of K. (λ_i are first order in x_1 and x_2). So $v(x) = \gamma\lambda_1(x)\lambda_2(x)\lambda_3(x)$ and γ is a number. Then we evaluate $v(x)$ at a^{123} to see that $\gamma = 0$. $\square$

Define the finite element space

$$V_h = \{v \in C^0(\bar{\Omega}) : v|_K \in P_3(K),\ \forall\, K \in \mathcal{T}_h\}.$$

Now, let $v_i \in P_3(K_i)$, $i = 1, 2$, where K_1 and K_2 are two triangles with common side S. If v_1 and v_2 take the same values at the end points and the two points a^{iij} of S, then $v_1 = v_2$ on S. So, we have proved that

$$\begin{aligned} V_h = \{v : v|_K \in P_3(K),\ \forall\, K \in \mathcal{T}_h, \\ v \text{ is continuous at the nodes and those } a^{iij} \text{ points}\}. \end{aligned}$$

Example 4 (Another P_3 on $\mathbb{R}^2$)

Theorem 5.4 *A function $v \in P_3(K)$ is uniquely determined by the degrees of freedom:*

$$\begin{aligned} &v(a^i), \qquad i = 1, 2, 3 \\ &\partial_{x_j} v(a^i), \qquad i = 1, 2, 3 \quad j = 1, 2 \\ &v(a^{123}) \end{aligned}$$

Proof: We only need to prove that $v(a^i) = \partial_{x_j} v(a^i) = v(a^{123}) = 0$ implies $v(x) = 0$. Then $v(x)$ vanishes at the 3 boundaries of K. Why? Because when taking derivative along the boundary,

$$\partial_s v(a^i) = s_1 \partial_{x_1} v(a^i) + s_2 \partial_{x_2} v(a^i).$$

The rest is the same as the proof of Theorem 5.3. $\square$

Define the finite element space

$$V_h = \{v : v|_K \in P_3(K),\ \forall\, K \in \mathcal{T}_h,\ \text{and } v, v_{x_i}, i = 1, 2 \text{ are continuous at the nodes}\}.$$

Example 5 (C^1 finite element on $\mathbb{R}^2$)

Theorem 5.5 *Let K be a triangle. A function $v \in P_5(K)$ is uniquely determined by the following degrees of freedom:*

$$\begin{aligned} &D^\alpha v(a^i), \qquad i = 1,2,3, \quad |\alpha| = \alpha_1 + \alpha_2 \le 2 \\ &\partial_n v(a^{ij}), \qquad i,j = 1,2,3, \quad i \ne j \end{aligned}$$

Proof: Again, we prove if all the dof are zero, then $v = 0$. If s is the direction along the boundary a^2a^3, then

$$v(a^i) = \partial_s v(a^i) = \partial_s^2 v(a^i) = 0, \qquad i = 2,3.$$

Since $v|_{a^2a^3}$ is a polynomial of degree at most 5, $v|_{a^2a^3} = 0$ and therefore $v(x) = u(x)\lambda_1(x)$.

Furthermore, we have

$$\partial_n v(a^{23}) = \partial_n v(a^i) = \partial_s \partial_n v(a^i) = 0, \qquad i = 2,3.$$

Since $(\partial_n v)|_{a^2a^3}$ is a polynomial of degree at most 4, $(\partial_n v)|_{a^2a^3} = 0$. By the product rule, $\partial_n v(x)|_{a^2a^3} = u(x)\partial_n \lambda_1(x)|_{a^2a^3} = cu(x)|_{a^2a^3} = 0$ with $c \ne 0$. So, $\lambda_1(x)$ is a factor of $u(x)$. Hence $v(x) = w(x)\lambda_1^2(x)$. In the same way, we see that we may also factor out $(\lambda_i(x))^2$ for $i = 2,3$. Hence $v = \gamma\lambda_1^2\lambda_2^2\lambda_3^3$. But $\lambda_i \in P_1(K)$ and $v \in P_5(K)$, the only possibility is $\gamma = 0$. $\square$

Define the finite element space

$$\begin{aligned} V_h = \{v : &v|_K \in P_5(K),\ \forall\, K \in \mathcal{T}_h, D^\alpha v \text{ is continuous at the nodes for } |\alpha| \le 2, \\ &\partial_n v \text{ is continuous at the mid points of each sides}\}. \end{aligned}$$

Now, let $v_i \in P_5(K_i)$, $i = 1,2$, where K_1 and K_2 are two triangles with common side S. If v_1 and v_2 satisfies

$$\begin{aligned} &D^\alpha v_1 = D^\alpha v_2 \qquad |\alpha| \le 2, \text{ at the endpoints of } S, \\ &\partial_n v_1 = \partial_n v_2 \qquad \text{at the midpoint of } S,\ n \text{ pointing from } K_1 \text{ to } K_2). \end{aligned}$$

Then the difference $w = v_1 - v_2$ satisfies

$$w = \partial_n w = 0 = \partial_s w \text{ on } S.$$

Hence $V_h \subset C^1(\bar{\Omega})$.

Example 6 (P_1 Lagrange element in $\mathbb{R}^3$) Let $\Omega \subset \mathbb{R}^3$ has the triangulation $\mathcal{T}_h = \{K\}$ of non-overlapping tetrahedrons K such that no vertex of one tetrahedron lies on a side of another tetrahedron. Define

$$P_r(K) = \{v : v(x) = \sum_{0 \le i+j+k \le r} b_{ijk} x_1^i x_2^j x_3^k\} \quad \text{when } K \subset \mathbb{R}^3.$$

Then

$$V_h = \{v \in C^0(\bar{\Omega}) : v|_K \in P_1(K), \quad \forall\, K \in \mathcal{T}_h\}.$$

with global dof being the values at the nodes of $\mathcal{T}_h$.

Example 7 (Rectangular finite element on $\mathbb{R}^2$. Tensor product element.) If $\Omega \subset \mathbb{R}^2$ is a square. Let K be a rectangle with vertices $a^1, ..., a^4$ and *with sides parallel to the coordinate axis in* $\mathbb{R}^2$. Define

$$Q_1(K) = \{v : v \text{ is bilinear on } K, \text{ i.e.} v(x)|_K = a_{00} + a_{10}x_1 + a_{01}x_2 + a_{11}x_1x_2\}.$$

Suppose Ω is divided into non-overlapping rectangles,

$$V_h = \{v \in C^0(\bar{\Omega}) : v|_K \in Q_1(K), \quad \forall\, K \in \mathcal{T}_h\}.$$

The values at the nodes can be used as global dof.

6 Approximation theory

Recall that for a typical elliptic problem, we have

$$\|u - u_h\|_V \leq \frac{\gamma}{\alpha}\|u - v_h\|_V, \qquad \forall v_h \in V_h. \tag{6.1}$$

So, one can choose v_h being the interpolant of u, namely $v_h = \pi_h u \in V_h$ to obtain an upper bound for the error $\|u - u_h\|$.

6.1 L^∞ interpolation error estimate with piecewise linear functions in 2d

Theorem 6.1 *Let K be a triangle with vertices a^i, i=1,2,3. Define h_K = the longest side of K and ρ_K = the diameter of the circle inscribed in K. Given $v \in C^0(K)$, let the interpolant $\pi v \in P_1(K)$ be defined by*

$$\pi v(a^i) = v(a^i), \qquad i = 1, 2, 3$$

Then,

$$\|v - \pi v\|_{L^\infty(K)} \leq 2h_K^2 \max_{|\alpha|=2} \|D^\alpha v\|_{L^\infty(K)}, \tag{6.2}$$

$$\max_{|\alpha|=1} \|D^\alpha(v - \pi v)\|_{L^\infty(K)} \leq 6\frac{h_K^2}{\rho_K} \max_{|\alpha|=2} \|D^\alpha v\|_{L^\infty(K)}, \tag{6.3}$$

where $\|v\|_{L^\infty(K)} = \max_{x\in K} |v(x)|$.

Proof: Let $\lambda_i(x)$ $i = 1, 2, 3$ be the basis functions for $P_1(K)$. Then

$$\pi v(x) = \sum_{i=1}^{3} v(a^i)\lambda_i(x). \tag{6.4}$$

Taylor expansion gives

$$v(y) = v(x) + \nabla v(x) \cdot (y - x) + R(x, y) \tag{6.5}$$

where

$$R(x, y) = \frac{1}{2}(y - x) \cdot D^2 v(\xi)(y - x) = \frac{1}{2}\sum_{i,j=1}^{2} \frac{\partial^2 v}{\partial x_i \partial x_j}(\xi)(y_i - x_i)(y_j - x_j).$$

Let $y = a_i$ in (6.5) and plug it into (6.4), we get

$$\pi v(x) = v(x) \sum_{i=1}^{3} \lambda_i(x) + \sum_{i=1}^{3} \left(\nabla v(x) \cdot (a^i - x) \right) \lambda_i(x) + \sum_{i=1}^{3} R(x, a^i) \lambda_i(x). \tag{6.6}$$

Note that

$$\max_i |R(x, a_i)| \leq 2h_k^2 \max_{|\alpha|=2} \|D^\alpha v\|_{L^\infty(K)} \tag{6.7}$$

Now, we need the following lemma, whose prove will be given later

Lemma 1 *Then for any function $v : \mathbb{R}^2 \supset K \to \mathbb{R}^1$ and for $j = 1, 2$,*

$$\sum_{i=1}^{3} \lambda_i(x) = 1, \qquad \partial_{x_j} \sum_{i=1}^{3} \lambda_i(x) = 0 \tag{6.8}$$

$$\sum_{i=1}^{3} \left(\nabla v(x) \cdot (a^i - x) \right) \lambda_i(x) = 0 \tag{6.9}$$

$$\sum_{i=1}^{3} \left(\nabla v(x) \cdot (a^i - x) \right) \partial_{x_j} \lambda_i(x) = \partial_{x_j} v(x). \tag{6.10}$$

Assuming this lemma, we have

$$\pi v(x) = v(x) + \sum_{i=1}^{3} R(x, a^i) \lambda_i(x).$$

Because $\lambda_i \geq 0$,

$$|\pi v(x) - v(x)| \leq \left(\sum_i \lambda_i \right) \left(\sup_{i, x \in K} |R(x, a^i)| \right) \leq 2h_k^2 \max_{|\alpha|=2} \|D^\alpha v\|_{L^\infty(K)}.$$

To prove (6.3), from (6.4)

$$\frac{\partial \pi v}{\partial x_1}(x) = \sum_i v(a^i) \frac{\partial \lambda_i}{\partial x_1}(x).$$

Again, let $y = a_i$ in (6.5) to get $v(a^i)$ and then plug it into the above equation. We get

$$\frac{\partial \pi v}{\partial x_1}(x) = v(x)\sum_i \frac{\partial \lambda_i}{\partial x_1}(x) + \sum_{i=1}^{3}\left(\nabla v(x)\cdot(a^i - x)\right)\partial_{x_1}\lambda_i(x) + \sum_i R(x,a^i)\frac{\partial \lambda_i}{\partial x_1}(x) \tag{6.11}$$

$$= \partial_{x_1} v(x) + \sum_i R(x,a^i)\frac{\partial \lambda_i}{\partial x_1}(x). \tag{6.12}$$

It is easy to see that

$$\max_{x\in K}|\partial_{x_1}\lambda_i(x)| \le \frac{1}{\rho_K}$$

The above equation together with (6.7) and (6.12) gives (6.3). $\square$

Proof of Lemma: $\sum_{i=1}^{3}\left(\nabla v(x)\cdot(a^i - x)\right)\lambda_i(x) = \nabla v(x)\cdot\left(\sum_{i=1}^{3}\lambda_i(x)(a^i - x)\right)$. So, we want to prove $\sum_{i=1}^{3}\lambda_i(x)(a^i - x) = 0$. Take any constant vector $b \in \mathbb{R}^2$ and let $v(x) = b\cdot x$. Then $\pi v = v$, $\nabla v(x) = b$ and $R(x,a^i) = 0$. Then (6.5) leads to

$$\sum_{i=1}^{3}\left(b\cdot(a^i - x)\right)\lambda_i(x) = 0.$$

To prove (6.10) for $j = 1$, we only need to show

$$\sum_{i=1}^{3}(a_1^i - x_1)\partial_{x_1}\lambda_i(x) = 1, \qquad \sum_{i=1}^{3}(a_2^i - x_2)\partial_{x_1}\lambda_i(x) = 0$$

Now we apply the same trick on (6.11) and get

$$b_1 = b_1\sum_i (a_1^i - x_1)\partial_{x_1}\lambda_i(x) + b_2\sum_i (a_2^i - x_2)\partial_{x_1}\lambda_i(x). \qquad \square$$

6.2 L^2 and H^1 interpolation error estimates

Theorem 6.2 *(error on one single element) With the same notation as in Theorem 6.1, there is an constant C such that*

$$\|v - \pi v\|_{L^2(K)} \le C h_K^2 |v|_{H^2(K)}, \tag{6.13}$$

$$|v - \pi v|_{H^1(K)} \le C\frac{h_K^2}{\rho_K}|v|_{H^2(K)}. \tag{6.14}$$

Theorem 6.3 *(error on Ω) Let $h = \max_{K\in\mathcal{T}_h} h_K$ and consider the triangulations $\mathcal{T}_h$ that satisfies*

$$\sup_{K\in Th} \frac{h_K}{\rho_K} \leq \frac{1}{\beta} \tag{6.15}$$

for some β independent of mesh size h. Then

$$\|u - \pi u\|^2_{L^2(\Omega)} = \sum_{K\in\mathcal{T}_h} \|u - \pi u\|^2_{L^2(K)} \leq Ch^4 |u|^2_{H^2(\Omega)}$$

$$|v - \pi v|^2_{H^1(\Omega)} \leq Ch^2 |v|^2_{H^2(\Omega)}.$$

Remark: : The condition (6.15) means that the triangles $K \in \mathcal{T}_h$ are not allowed to be arbitrarily thin, or equivalently, the angles of the triangles K are not allowed to be arbitrarily small.

We can generalize Theorem 6.2 to finite element of polynomial with degree $r \geq 1$:

Theorem 6.4 *Consider piecewise polynomials of degree $r \geq 1$ on triangle K.*

$$\|v - \pi v\|_{L^2(K)} \leq Ch_K^{r+1} |v|_{H^{r+1}(K)}, \tag{6.16}$$

$$|v - \pi v|_{H^1(K)} \leq Ch_K^{r} |v|_{H^{r+1}(K)}. \tag{6.17}$$

If we are given Ω with an triangulation $\mathcal{T}_h$ satisfying (6.15), we apply (6.16) and (6.17) on each $K \in \mathcal{T}_h$ and then add them up to obtain the global interpolation error estimate

$$\|v - \pi v\|_{L^2(\Omega)} \leq Ch^{r+1} |v|_{H^{r+1}(\Omega)}, \quad |v - \pi v|_{H^1(\Omega)} \leq Ch^{r} |v|_{H^{r+1}(\Omega)} \tag{6.18}$$

Sketched proof of Theorem 6.4: (Bramble-Hilbert + scaling) Step I:

Lemma 2 *(Bramble-Hilbert Lemma. Generalization of Poincare inequality) Consider domain Σ with Lipschitz continuous boundary, there is a constant C only depends on Σ such that*

$$\inf_{p\in P_r(\Sigma)} \|v - p\|_{H^{r+1}(\Sigma)} \leq C|v|_{H^{r+1}(\Sigma)}$$

Proof: See for example Ciarlet page 115 which is for more general Banach space $W^{r+1,p}(\Sigma)$. It is proved by contradiction and proof uses Rellich Lemma (compact embedding). $\square$

Now, Suppose Σ is a triangle of size 1. Note that the interpolation operator π is a bounded operator from $C^\ell(\Sigma)$ to $H^{r+1}(\Sigma)$ ($\ell = 0$ for C^0 element, $\ell = 1$ for C^1 element). When $r+1$ is large enough such that $H^{r+1}(\Sigma) \subset C^\ell(\Sigma)$. (When $\ell = 0$, $r = 1$ is enough for $\Sigma \subset \mathbb{R}^n$ with $n = 1, 2, 3$.) So,

$$\begin{aligned}
\|(I-\pi)u\|_{H^{r+1}(\Sigma)} &= \inf_{p\in P_r} \|(I-\pi)(u-p)\|_{H^{r+1}(\Sigma)} \\
&\le \inf_{p\in P_r} \left(\|u-p\|_{H^{r+1}(\Sigma)} + C\|u-p\|_{C^\ell(\Sigma)}\right) \\
&\le C \inf_{p\in P_r} \|u-p\|_{H^{r+1}(\Sigma)} \le C|v|_{H^{r+1}(\Sigma)}.
\end{aligned}$$

In particular,

$$\|(I-\pi)u\|_{L^2(\Sigma)} \le C_0|v|_{H^{r+1}(\Sigma)}, \quad |(I-\pi)u|_{H^1(\Sigma)} \le C_0|v|_{H^{r+1}(\Sigma)} \tag{6.19}$$

with C_0 a constant dependent on Σ.

Step II: use *scaling argument to obtain the factor* h^{r+1}. Let's see a specific example:

Let Σ be the triangle with vertices $(0,0), (0,1), (1,0)$ and let K be the triangle with vertices $(0,0), (0,h), (h,0)$. Suppose the interpolation uses the function values at the some points on Σ, which will be called $\{..., N_i, ...\}$. For P_r polynomial, that's $\frac{(r+1)(r+2)}{2}$ points. The following approach also applies when interpolation uses derivative values. Suppose $u(x_1, x_2)$ is defined on K. Define $v(x_1, x_2) = u(hx_1, hx_1)$. Then $v(x_1, x_2)$ is defined on Σ.

$$\begin{aligned}
&\|(I-\pi)v(x_1,x_2)\|^2_{L^2(\Sigma)} = \int_\Sigma |v(x_1,x_2) - \sum_j v(N_j)\varphi_j(x_1,x_2)|^2 dx_1 dx_2 \\
&= \frac{1}{h^2}\int_\Sigma |u(hx_1,hx_2) - \sum_j u(hN_j)\varphi_j(x_1,x_2)|^2 d(hx_1)d(hx_2) \\
&= \frac{1}{h^2}\int_K |u(y_1,y_2) - \sum_j u(hN_j)\varphi_j(y_1/h, y_2/h)|^2 dy_1 dy_2 \\
&= \frac{1}{h^2}\int_K |(I-\pi)u(y_1,y_2)|^2 dy_1 dy_2
\end{aligned}$$

$$\|D_x^\alpha(I-\pi)v(x_1,x_2)\|^2_{L^2(\Sigma)} = \int_\Sigma |D_x^\alpha(I-\pi)u(hx_1,hx_2)|^2 dx_1dx_2$$
$$= \frac{1}{h^2}\int_\Sigma |h^{|\alpha|}D^\alpha_{hx}u(hx_1,hx_2) - \sum_j u(hN_j)D_x^\alpha\varphi_j(x_1,x_2)|^2 d(hx_1)d(hx_2)$$
$$= \frac{1}{h^2}\int_K |h^{|\alpha|}D_y^\alpha u(y_1,y_2) - h^{|\alpha|}\sum_j u(hN_j)D_y^\alpha\varphi_j(y_1/h,y_2/h)|^2 dy_1dy_2$$
$$= \frac{h^{2|\alpha|}}{h^2}\int_K |D_y^\alpha(I-\pi)u(y_1,y_2)|^2 dy_1dy_2.$$

Similarly,

$$\|D_x^\alpha v\|^2_{L^2(\Sigma)} = \frac{h^{2|\alpha|}}{h^2}\int_K |D_y^\alpha u(y_1,y_2)|^2 dy_1dy_2.$$

So, (6.16) and (6.17) follow from (6.19) after the scaling. Indeed, you see that this approach can generate more general results than (6.16) and (6.17). For example, interpolation error in $W^{s,p}(\Omega)$. See Ciarlet or Brenner&Scott. □

Question: where does the ration h_K/ρ_K come into play? Suppose K is the triangle with vertices $(0,0),(0,h),(\beta h,0)$ for some β small. You repeat the above scaling argument, then you find that the factor is still $h^{|\alpha|}$ because you will get factors $\beta^s h^{|\alpha|}$ and $h^{|\alpha|}$ at the same time, and the larger factor $h^{|\alpha|}$ dominates. What it implies? It means the interpolation error on triangle $(0,0),(0,h),(h,0)$ and on triangle $(0,0),(0,h),(\beta h,0)$ are almost the same, even though the latter is much smaller, we do not gain! Remember that with a fixed Ω, we will need much more triangle of the 2nd kind to fill it. The size of the problem is increased, while the accuracy remains the same!

Remark: : If u does not have enough regularity (i.e., not in $H^{r+1}(\Omega)$), we have the corresponding reduction in the power of h: For $1 \le s \le r+1$, we have

$$\|v-\pi v\|_{L^2(\Omega)} \le Ch^s|v|_{H^s(\Omega)}, \quad |v-\pi v|_{H^1(\Omega)} \le Ch^{s-1}|v|_{H^s(\Omega)}. \tag{6.20}$$

6.3 examples

Example 1 Let $V_h = \{v \in C^0(\bar\Omega) : v|_K \in P_2(K),\ \forall K \in \mathcal{T}_h\}$. Then (6.18) holds with $r=2$.

Example 2 Let $V_h = \{v \in C^1(\bar{\Omega}) : v|_K \in P_5(K),\ \forall K \in \mathcal{T}_h\}$ being the C^1 finite element of **Example 5** in Section 5. Let the interpolation $\pi v \in V_h$ is given by

$$D^\alpha \pi v = D^\alpha v \text{ at the nodes of } \mathcal{T}_h,\ |\alpha| \le 2$$
$$\partial_n \pi v = \partial_n v \text{ at the modpoints of each side } S \text{ of } \mathcal{T}_h.$$

Then (6.18) holds with $r = 5$.

7 Error estimates for elliptic problem

Recall that the typical error estimate for an elliptic problem is

$$\|u - u_h\|_V \leq C \inf_{v_h \in V_h} \|u - v_h\|_V \leq C\|u - \pi u\|_V.$$

Example 1 For $V \subset H^1$ (Section 4.2, **Example 1,2**) and

$$V_h = \{v \in V : v|_K \in P_r(K),\ \forall\, K \in \mathcal{T}_h\},$$

we have

$$\|u - u_h\|_{H^1(\Omega)} \leq Ch^r |u|_{H^{r+1}(\Omega)}.$$

Example 2 For the biharmonic problem (Section 3 **Example 3,4**) with C^1 finite element (Section 4.2, **Example 3,4**), we have

$$\|u - u_h\|_{H^2(\Omega)} \leq Ch^4 |u|_{H^6(\Omega)}.$$

7.1 regularity of exact solution

Consider Poisson equation

$$-\Delta u = f \text{ in } \Omega, \qquad u|_\Gamma = 0.$$

When Γ is a smooth curve, for $s = 0, 1, ...,$, we have

$$\|u\|_{H^{s+2}(\Omega)} \leq C\|f\|_{H^s(\Omega)}. \tag{7.1}$$

If Γ is not smooth, (7.1) may not hold, not even for $s = 0$. Consider the domain $\Omega = \{(r\cos(\theta), r\sin(\theta)) : 0 < r < 1, 0 < \theta < \omega\}$ be a pie-shaped domain of angle ω. In polar coordinates

$$\Delta u = \frac{1}{r}\frac{\partial}{\partial r}\left(r\frac{\partial u}{\partial r}\right) + \frac{1}{r^2}\frac{\partial^2 u}{\partial \theta^2}.$$

Let $u = r^\gamma \sin(\gamma\theta)$ with $\gamma = \frac{\pi}{\omega}$. Then it is easy to check that

$$\Delta u = 0, \qquad u|_{\theta=0,\omega} = 0.$$

However, when $\gamma < 1$, u is not in $H^2(\Omega)$. On the other hand, one can show that (7.1) holds with $s = 0$ if Ω is a convex polygonal domain.

Example 1 For a solution u of the form

$$u = r^\gamma \alpha(\theta) + \beta(r, \theta), \qquad \gamma = \frac{\pi}{\omega}$$

we have formally that

$$u \in H^s(\Omega) \Longleftrightarrow \int_\Omega |D^s u|^2 dx \sim C \int_0^R |r^{\gamma - s}|^2 r dr < \infty$$

Hence $u \in H^s(\Omega)$ if and only if $s < \gamma + 1$. Therefore, for the standard finite element method for Poisson equation in a polygonal domain, for any $\varepsilon > 0$,

$$\|u - u_h\|_{H^1(\Omega)} \leq C h^{\gamma - \varepsilon} \|u\|_{H^{\gamma + 1 - \varepsilon}(\Omega)}$$

where $\gamma = \frac{\pi}{\omega}$ and ω is the maximal angle of a corner of Γ.

7.2 an error estimate in the $L^2(\Omega)$ norm

Theorem 7.1 *If Ω is a convex polygonal domain and u_h is the finite element solution of the Poisson equation. Let $e = u - u_h$, then*

$$\|e\|_{L^2(\Omega)} \leq Ch \|e\|_{H^1(\Omega)} \leq Ch^{r+1} \|u\|_{H^{r+1}(\Omega)}.$$

Proof: Let $-\Delta\psi = e$, $\psi|_\Gamma = 0$. Then $\|\psi\|_{H^2} \leq C\|e\|_{L^2}$. Recall $a(e, v_h) = (\nabla e, \nabla v_h) = 0$ for any $v_h \in V_h$. So,

$$\begin{aligned}(e, e) &= (e, -\Delta\psi) = (\nabla e, \nabla\psi) = (\nabla e, \nabla(\psi - \pi\psi)) \\ &\leq \|\nabla e\|_{L^2} \|\nabla(\psi - \pi\psi)\|_{L^2} \leq C\|\nabla e\|_{L^2} h \|\Delta\psi\|_{L^2} \leq Ch\|\nabla e\|_{L^2} \|e\|_{L^2}. \qquad \square\end{aligned}$$

7.3 inverse inequality and the condition number of the stiffness matrix

When we use an iterative method to solve for $A\xi = b$, the rate of convergence depends on the condition number $\kappa(A)$ of A, where $\kappa(A)$ is defined as the ratio of the maximum and minimum eigenvalues of A:

$$\kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

For example, the conjugate gradient method satisfies

$$\|\xi_k - \xi\|_A \leq 2\left(\frac{\sqrt{\kappa(A)}-1}{\sqrt{\kappa(A)}+1}\right)^k \|\xi_0 - \xi\|_A$$

where $\|\eta\|_A^2 = \eta \cdot A\eta$. So, the larger the condition number, the slower the convergence. Now, we are going to use a useful inequality — the inverse inequality to prove that for the Poisson equation, $\kappa(A) = O(h^{-2})$.

Lemma 3 *(Inverse inequality) For the quasi-uniform mesh, there is a constant C such that for any $v_h \in V_h$*

$$\|\nabla v_h\|_{L^2(\Omega)} \leq Ch^{-1}\|v_h\|_{L^2(\Omega)}. \tag{7.2}$$

Proof: (I) By the scaling argument, we only need to show on on a triangle K of size 1, $(h = 1)$

$$\|\nabla v_h\| \leq C\|v_h\| \tag{7.3}$$

for some C independent of v_h. (II) When $h = O(1)$, (7.3) is obvious which follows from something like $(a-b)^2 < 2(a^2+b^2)$. $\square$

Question: Do we have

$$\|v_h\|_{L^2(\Omega)} \leq Ch\|\nabla v_h\|_{L^2(\Omega)}? \tag{7.4}$$

No. (Find a counter example.) Then, what is wrong with scaling argument? Because even on a triangle of size 1, we do not have $\|v_h\| \leq \|\nabla v_h\|$ unless we have zero boundary condition. But with this constraint, taken any function v on Ω, the restriction of v might not satisfy the zero boundary condition on each small triangle.

Now, let us consider the condition number of A. For any vector η, define $v_h(x) = \sum_i \eta_i \varphi_i(x) \in V_h$. It is rather obvious that

Lemma 4

$$C_1 h^2|\eta|^2 \leq \int_\Omega |v_h|^2 dx \leq C_2 h^2|\eta|^2.$$

Proof: If it is not obvious to you, first prove it on triangle of size 1, then use scaling argument to obtain the factor h^2. $\square$

So,

$$\frac{\eta \cdot A\eta}{|\eta|^2} = \frac{a(v_h, v_h)}{|\eta|^2} = \frac{\|\nabla v_h\|^2}{|\eta|^2}$$
$$\frac{ch^2|\eta|^2}{|\eta|^2} \leq \frac{c\|v_h\|^2}{|\eta|^2} \leq \frac{\|\nabla v_h\|^2}{|\eta|^2} \leq \frac{Ch^{-2}\|v_h\|^2}{|\eta|^2} \leq \frac{C|\eta|^2}{|\eta|^2}$$
$$\implies \lambda_{\max} < C, \qquad \lambda_{\min} > ch^2$$

Therefore $\kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}} \leq Ch^{-2}$.

8 Parabolic problem

Consider

$$\partial_t u - \Delta u = f \qquad \text{in } \Omega \times I \tag{8.1}$$
$$u = 0 \qquad \text{on } \Gamma \times I \tag{8.2}$$
$$u(0,x) = u^0(x). \tag{8.3}$$

8.1 semi-discretization in space

Let $V = H_0^1(\Omega)$. find $u(t) \in V$, $t \in I$ such that

$$(\partial_t u(t), v) + a(u(t), v) = (f(t), v) \qquad \forall v \in V, t \in I \tag{8.4}$$
$$u(0,x) = u^0(x). \tag{8.5}$$

Replace V by the finite dimensional subspace V_h, we get

$$(\partial_t u_h(t), v_h) + a(u_h(t), v_h) = (f(t), v_h) \qquad \forall v_h \in V_h, t \in I \tag{8.6}$$
$$(u_h(0), v_h) = (u^0, v_h) \qquad \forall v_h \in V_h. \tag{8.7}$$

Let us use the representation

$$u_h(t,x) = \sum_{i=1}^{M} \xi_i(t)\varphi_i(x)$$

with the time dependent coefficients $\xi_i(t)$. Then, let $v_h = \varphi_j$

$$\sum_{i=1}^{M} \dot{\xi}_i(t)(\varphi_i, \varphi_j) + \sum_{i=1}^{M} \xi_i(t) a(\varphi_i, \varphi_j) = (f(t), \varphi_j) \qquad j = 1, ..., M, t \in I \tag{8.8}$$

$$\sum_{i=1}^{M} \xi_i(0)(\varphi_i, \varphi_j) = (u^0, \varphi_j). \tag{8.9}$$

or in matrix form

$$B\dot{\xi}(t) + A\xi(t) = F(t), \qquad t \in I \tag{8.10}$$
$$B\xi(0) = U^0. \tag{8.11}$$

This is a system of ODEs. The solution is

$$\xi(t) = e^{-B^{-1}At}B^{-1}U^0 + \int_0^t e^{-B^{-1}A(t-s)}B^{-1}F(s)ds, \qquad t \in I.$$

However, because $B^{-1}A$ is not symmetric, we will not deal with this system directly. Instead, we do a decomposition

$$B = \Omega^\tau \Lambda \Omega$$

with Ω being an orthonormal matrix and Λ an diagonal matrix. Of course, every diagonal entry of Λ is positive. Moreover, we claim that they are all of order h^2.

Proof: For any $\eta = (\eta_1, ..., \eta_M)$, let $v_h(x) = \sum_i \eta_i \varphi_i(x) \in V_h$. Then

$$\begin{aligned}
&\frac{\eta \cdot B\eta}{|\eta|^2} = \frac{\|v_h\|^2}{|\eta|^2} \\
&\frac{ch^2|\eta|^2}{|\eta|^2} \le \frac{\|v_h\|^2}{|\eta|^2} \le \frac{Ch^2|\eta|^2}{|\eta|^2} \\
&\Longrightarrow \lambda_{\max}(B) < Ch^2, \quad \lambda_{\min}(B) > ch^2. \qquad \square
\end{aligned}$$

Let

$$E = \Omega^\tau \Lambda^{1/2} \Omega. \tag{8.12}$$

Then $B = E^\tau E$. Let $\eta = E\xi$, then (8.10)–(8.11) becomes

$$\dot{\eta} + \bar{A}\eta = \bar{F} \tag{8.13}$$

$$\eta(0) = \eta^0. \tag{8.14}$$

where $\bar{A} = E^{-\tau}AE^{-1}$, $\bar{F} = E^{-\tau}F$, $\eta^0 = E^{-\tau}U^0$. The solution of (8.13) is

$$\eta(t) = e^{-\bar{A}t}\eta^0 + \int_0^t e^{-\bar{A}(t-s)}\bar{F}(s)ds, \qquad t \in I.$$

Suppose $F = 0$, then

$$\dot{\eta}(t) = \frac{-1}{t}\left((\bar{A}t)\, e^{-\bar{A}t}\right)\eta^0.$$

Since $\sup_{x>0} xe^{-x} = O(1)$, we have

$$|\bar{A}\eta(t)| = |\dot{\eta}(t)| \le \frac{C|\eta^0|}{t}. \tag{8.15}$$

Here we have used the property that $\bar{A}$ is symmetric and all eigenvalues are non-negative. Now, using $\|\bar{A}\| \le \|E^{-\tau}\|\|A\|\|E^{-1}\| \le Ch^{-2}$ and $\|e^{-\bar{A}t}\| \le 1$ for $t \ge 0$,

$$\int_0^T |\dot{\eta}(t)|dt \le \int_0^{h^2} \frac{C|\eta(0)|}{h^2} + \int_{h^2}^T \frac{C|\eta(0)|}{t} dt \le C\left(1 + \left|\log(T/h^2)\right|\right)|\eta(0)|. \tag{8.16}$$

Now, because $\eta = E\xi$ and because E is bounded from above and from below by quantities of the same order, namely Ch, from (8.16), we finally obtain the following estimates of the original system (8.10)–(8.11)

$$\int_0^T |\dot{\xi}(t)|dt \le C\left(1 + \left|\log(T/h^2)\right|\right)|\xi(0)|. \tag{8.17}$$

Recall that $\lambda_{\max}(A) = O(1)$. So, $\lambda_{\max}(\bar{A}) = O(h^{-2})$ and hence the ODE system (8.13)–(8.14) is very stiff! Namely, if you used for example forward Euler to solve it, the time step has to be very small. Recall that if we use forward Euler to solve for $y_t + \lambda y = 0$ with $\lambda > 0$, we get

$$\frac{y^{n+1} - y^n}{\Delta t} + \lambda y^n = 0, \qquad y^{n+1} = (1 - \lambda\Delta t)y^n$$

and hence the stability requires that $|1 - \lambda\Delta t| \le 1$. When $\lambda \gg 1$, Δt as to be small, although the exact solution is $y_0 e^{-\lambda t}$ which is very very small when t large. Implicit method like backward Euler can overcome this problem.

Back to our finite element method. Suppose $f = 0$. Let $v_h = u_h(t)$ in (8.6), we get

$$(\partial_t u_h(t), u_h(t)) + a(u_h(t), u_h(t)) = 0 = \frac{1}{2}\frac{d}{dt}\|u_h(t)\|^2 + a(u_h(t), u_h(t)) = 0$$

So,

$$\|u_h(t)\|^2 + 2\int_0^t a(u_h(t), u_h(t)) = \|u_h(0)\|^2 \le \|u^0\|^2.$$

Theorem 8.1 *There is a constant C such that*

$$\max_{t\in I} \|u(t) - u_h(t)\| \le C\left(1 + |\log(T/h^2)|\right) \max_{t\in I} h^2 \|u(t)\|_{H^2(\Omega)}.$$

Proof: First, we project u onto V_h:

$$a(u(s) - \tilde{u}_h(s), v_h) = 0, \qquad \forall v_h \in V_h,\ s \in (0, T). \tag{8.18}$$

Instead of working on $u - u_h$, we will work on $e_h(t) := \tilde{u}_h(t) - u_h(t)$ since (1) $e_h(t) \in V_h$. (2) we already know $\theta(t) = \tilde{u}_h(t) - u(t)$ is small: $\|\tilde{u}_h - u\| \leq Ch^2\|u(t)\|_{H^2(\Omega)}$.

So, we want to estimate $\|e_h(t)\|_{L^2}^2 = (e_h(t), e_h(t))$. How can we obtain this term? We know that for any $\varphi_h(t)$, we have

$$(\dot{e}_h(t), \varphi_h(t)) + a(e_h(t), \varphi_h(t)) = \frac{d}{dt}(e_h(t), \varphi_h(t)) - (e_h(t), \dot{\varphi}_h(t)) + a(e_h(t), \varphi_h(t)). \tag{8.19}$$

Integrating on t, we can obtain $(e_h(t), \varphi_h(t))$. So, naturally, we want

$$\varphi_h(t) = e_h(t) \tag{8.20}$$

The left hand side of (8.19) is already very small. So we require the rest term on right hand side is also small, since we want to prove the smallness of $(e_h(t), \varphi_h(t))(= \|e_h(t)\|^2)$. Hence we require

$$-(e_h(s), \dot{\varphi}_h(s)) + a(e_h(s), \varphi_h(s)) = 0, \qquad \forall s \in (0, t). \tag{8.21}$$

So, $\varphi_h(s)$ satisfies a backward heat equation and that is also the reason why we want to work on an $e_h \in V_h$.

So, define $\varphi_h : (0, t) \to V_h$ by (8.20) and

$$-(v_h, \dot{\varphi}_h(s)) + a(v_h, \varphi_h(s)) = 0, \qquad \forall\ v_h \in V_h,\ s \in (0, t) \tag{8.22}$$

which implies (8.21).

Then integrating on (8.19) from 0 to t, we get

$$\begin{aligned}
(e_h(t), \varphi_h(t)) &= (e_h(0), \varphi_h(0)) + \int_0^t (\dot{e}_h(s), \varphi_h(s)) + a(e_h(s), \varphi_h(s))ds \\
&= (\theta(0), \varphi_h(0)) + \int_0^t (\dot{\theta}(s), \varphi_h(s)) + a(\theta(s), \varphi_h(s))ds \\
&= (\theta(0), \varphi_h(0)) + \int_0^t (\dot{\theta}(s), \varphi_h(s))ds
\end{aligned}$$

We have used $(\theta(0) - e_h(0), \varphi_h(0)) = 0$ and (8.18) in the above derivation. Then using integration by part again, we get

$$\|e_h(t)\|^2 \leq -\int_0^t (\theta(s), \dot{\varphi}_h(s))ds + (\varphi_h(t), \theta(t)) \tag{8.23}$$

Exactly like how we prove (8.15) and (8.17), and using $\|\varphi\|^2_{L^2(\Omega)} \sim h^2|\eta|^2_{\ell^2}$ (Section 7 Lemma 4), we can prove

$$\int_0^t \|\dot{\varphi}_h(s)\|ds \leq C\left(1 + \left|\log(t/h^2)\right|\right)\|e_h(t)\|.$$

Putting all these together and using Cauchy-Schwarz on (8.23), we get

$$\|e_h(t)\| \leq C\left(1 + \left|\log(t/h^2)\right|\right) \max_{s\in[0,t]} \|\theta(s)\|. \qquad \square$$

8.2 backward Euler

Use backward Euler on the ODE system (8.6), we get

$$(\frac{u_h^n - u_h^{n-1}}{k_n}, v_h) + a(u_h^n, v_h) = (f(t_n), v_h) \qquad \forall v_h \in V_h \tag{8.24}$$

$$(u_h^0, v_h) = (u^0, v_h) \qquad \forall v_h \in V_h. \tag{8.25}$$

Let $v_h = u_h^{n+1}$, we get

$$\frac{1}{2k_n}\left(\|u_h^n\|^2 - \|u_h^{n-1}\|^2 + \|u_h^n - u_h^{n-1}\|^2\right) + \|\nabla u_h^n\|^2 = (f(t_n), u_h^n).$$

If $f = 0$, we get $\|u_h^n\| \leq \|u_h^{n-1}\| \leq ... \leq \|u_h^0\|^2 \leq \|u^0\|^2$. What if $f \neq 0$? Use Cauchy-Schwarz and Poincare

$$(f(t_n), u_h^n) \leq \|f(t_n)\|\|u_h^n\| \leq C\|f(t_n)\|\|\nabla u_h^n\| \leq C\|f(t_n)\|^2 + \frac{1}{2}\|\nabla u_h^n\|^2.$$

So,

$$\frac{1}{2}\left(\|u_h^n\|^2 - \|u_h^{n-1}\|^2 + \|u_h^n - u_h^{n-1}\|^2\right) + \frac{1}{2}\|\nabla u_h^n\|^2 k_n \leq C\|f(t_n)\|^2 k_n.$$

Sum up, we get

$$\|u_h^n\|^2 + \sum_i \frac{1}{2}\|\nabla u_h^i\|^2 k_i \leq \|u_h^0\|^2 + C\sum_i \|f(t_n)\|^2 k_n.$$

So, we have unconditionally stability.

8.3 Crank-Nicolson

We can also have an unconditional stable 2nd order scheme: (Taylor expand at $t^{n-1/2}$ to see it is 2nd order in time)

$$(\frac{u_h^n - u_h^{n-1}}{k_n}, v_h) + a\left(\frac{u_h^n + u_h^{n-1}}{2}, v_h\right) = \left(\frac{f(t_n) + f(t_{n-1})}{2}, v_h\right) \qquad \forall v_h \in V_h \tag{8.26}$$

$$(u_h^0, v_h) = (u^0, v_h) \qquad \forall v_h \in V_h. \tag{8.27}$$

Let $v_h = \frac{u_h^n + u_h^{n-1}}{2}$ to see that it is unconditional stable.

Remark: By Lax equivalence theorem, stability + consistency $\Rightarrow$ convergence. How do you prove error accuracy? (Hint: $u(t)$ satisfies (8.24) or (8.26) with a local truncation error term. Define the projection of $u(t^n)$, call it $\tilde{u}_h^n$, by $a(\tilde{u}_h^n, v_h) = a(u(t^n), v_h)$. What happens if you use $(\tilde{u}_h^n, v_h) = (u(t^n), v_h)$. What equation does $\tilde{u}_h^n$ satisfy? Same equation but with some additional error terms. Then compare u_h^n with $\tilde{u}_h^n$. Comparing that with stability proof, the main difference is that the forcing term $f(t^n)$ is gone, or we say it is replaced by the local error term. Unlike $f(t^n)$, those terms are small and then being added up to form the global error.)

8.4 discontinuous Galerkin

Introduce

$$W_{hk} = \{v : I \to V_h : v|_{I_n} \in P_q(I_n), n = 1, ..., N\} \tag{8.28}$$

where

$$P_q(I_n) = \{v : I \to V_h : v(t) = \sum_{i=0}^{q} v_i t^i \text{ with } v_i \in V_h\}. \tag{8.29}$$

Note that the functions v in W_{hk} may be discontinuous in time at the discrete time level t_n. So we introduce the notation

$$v_+^n = \lim_{s \to 0+} v(t_n + s), \quad v_-^n = \lim_{s \to 0-} v(t_n + s), \qquad [v^n] = v_+^n - v_-^n. \tag{8.30}$$

Suppose $I_n = [t_{n-1}, t_n]$. Let $U_-^0 = u^0$. For $n = 1, ..., N$, given U_-^{n-1}, find $U = U|_{I_n} \in P_q(I_n)$ such that for any $v \in P_q(I_n)$

$$\int_{I_n} \Big((\dot{U}, v) + a(U, v)\Big)\, dt + (U_+^{n-1}, v_+^{n-1}) = \int_{I_n} (f, v) dt + (U_-^{n-1}, v_+^{n-1}) \tag{8.31}$$

where $U_-^0 = u^0$. For $q = 0$, U is constant in time on each I_n. Using the notation $U^n = U_-^n = U_+^{n-1}$, (8.31) reduces to

$$k_n a(U^n, v) + (U^n, v) = \int_{T_n} (f, v)dt + (U^{n-1}, v), \qquad \forall\, v \in V_h \tag{8.32}$$

where $U^0 = u^0$. This is a simple modification of the backward Euler scheme.

For $q = 1$, we assume on $I_n = [t_{n-1}, t_n]$, $U(t) = U_0 + \frac{t-t_{n-1}}{k_n} U_1$, $U_i \in V_h$. Plugging it into (8.31),

$$\int_{I_n} \left((\frac{U_1}{k_n}, v + \frac{t - t_{n-1}}{k_n} w) + a(U_0 + \frac{t - t_{n-1}}{k_n} U_1, v + \frac{t - t_{n-1}}{k_n} w) \right) dt$$
$$+ (U_0, v) = \int_{I_n} (f, v + \frac{t - t_{n-1}}{k_n} w)dt + (U_-^{n-1}, v)$$

Note that $\int_{t_{n-1}}^{t_n} \frac{t-t_{n-1}}{k_n} dt = \frac{k_n}{2}$, $\int_{t_{n-1}}^{t_n} \left(\frac{t-t_{n-1}}{k_n}\right)^2 dt = \frac{k_n}{3}$.

$$(U_1, v) + \frac{1}{2}(U_1, w) + k_n a(U_0, v) + \frac{k_n}{2} a(U_0, w) + \frac{k_n}{2} a(U_1, v) + \frac{k_n}{3} a(U_1, w)$$
$$+ (U_0, v) = \int_{I_n} (f, v + \frac{t - t_{n-1}}{k_n} w)dt + (U_-^{n-1}, v)$$

Let $w = 0$, we get

$$(U_1, v) + k_n a(U_0, v) + \frac{k_n}{2} a(U_1, v) + (U_0, v) = \int_{I_n} (f, v)dt + (U_-^{n-1}, v), \qquad \forall v \in V_h \tag{8.33}$$

Let $v = 0$, we get

$$\frac{1}{2}(U_1, w) + \frac{k_n}{2} a(U_0, w) + \frac{k_n}{3} a(U_1, w) = \int_{I_n} (f, \frac{t - t_{n-1}}{k_n} w)dt, \qquad \forall w \in V_h \tag{8.34}$$

The above two equations are the scheme for $q = 1$.

8.4.1 unconditional stable when $f = 0$

Now, suppose $f = 0$ and let $v = U$ in (8.31), we have

$$\frac{1}{2}\left(\|U_-^n\|^2 - \|U_+^{n-1}\|^2\right) + \int_{I_n} a(U, U)dt + \|U_+^{n-1}\|^2 \le \frac{1}{2}\|U_-^{n-1}\|^2 + \frac{1}{2}\|U_+^{n-1}\|^2.$$

So,

$$\|U_-^n\| \le \|U_-^{n-1}\|. \tag{8.35}$$

Wow! This is *unconditional* stability for scheme of *any order* in time! This can also be viewed as a motivation for the design of the scheme (8.31).

8.4.2 stability of discrete time derivative for the $q = 0$ and $f = 0$ case

The results of this section is needed in the proof of error estimates later. As we will see, we only need to study the $f = 0$ case here. First let $v = U^n$ in (8.32), we obtain

$$\|U^n\| \le \|U^{n-1}\|, \qquad \sum_n \frac{1}{2}\|U^n - U^{n-1}\|^2 + \sum_{n=1}^N k_n \|\nabla U^n\|^2 \le \|U^0\|. \tag{8.36}$$

Then let $v = t^n(U^n - U^{n-1})$ in (8.32), we get

$$t^n \left\| \frac{U^n - U^{n-1}}{k_n} \right\|^2 k_n + t^n a(U^n - U^{n-1}, U^n) = 0 \tag{8.37}$$

Since

$$t^n a(U^n - U^{n-1}, U^n) = \frac{t^n}{2}\left(\|\nabla U^n\|^2 - \|\nabla U^{n-1}\|^2 + \|\nabla(U^n - U^{n-1})\|^2\right),$$

adding up (8.37) and dropping the postive terms $\frac{t^n}{2}\|\nabla U^n\|^2$ and $\sum_n \frac{t^n}{2}\|\nabla(U^n - U^{n-1})\|^2$, we get

$$\sum_n t^n \left\| \frac{U^n - U^{n-1}}{k_n} \right\|^2 k_n \le \sum_n k_{n+1}\|\nabla U^n\|^2 \le C\|U^0\|^2. \tag{8.38}$$

if we assume $k_{n+1} \le C k_n$.

$$\begin{aligned} \sum_n \left\| \frac{U^n - U^{n-1}}{k_n} \right\| k_n &\le \left(\sum_n t^n \left\| \frac{U^n - U^{n-1}}{k_n} \right\|^2 k_n \right)^{1/2} \left(\sum_{n=1}^N \frac{k_n}{t^n} \right)^{1/2} \\ &\le C\|U^0\| \left(1 + \log \frac{T}{k_1}\right)^{1/2}. \end{aligned} \tag{8.39}$$

8.4.3 error estimate for the $q = 0$ case

Theorem 8.2 *($q = 0$ scheme) Suppose there is a constant γ such that the time step k_n satisfy $k_{n-1} \le \gamma k_n$. Then there is a constant C depends only on γ and $\beta = \inf_{K\in\mathcal{T}_h} \frac{\rho_K}{h_K}$ such that*

$$\|u(t_n)-U^n|\| \le C\left(1+\log\frac{t_n}{k_n}\right)^{1/2}\left(\max_{m\le n}\int_{I_m}\|\dot{u}(s)\|ds+\max_{t\le t_n}h^2\|u(t)\|_{H^2(\Omega)}\right)$$

Remark: Comparing the theorem with the results we have proved before (See the remark of last section), we see that here the bounds depends only on $\|u\|_{H^2}$ and $\|\dot{u}\|_{L^2}$. The bounds in our previous results depends on $\|\dot{u}\|_{H^2(\Omega)}$ and hence is not optimal even though it does not have the $-\log k_n$ term. Is it a big deal? Yes! The optimal bounds in here can be applied to control the step size!

Proof: I. Introduce the interpolant $\tilde{U} \in W_{hk}$ defined by

$$\int_{I_n} a(\tilde{U}-u,v)ds = 0, \qquad \forall v \in W_{hk},\ n = 1,...,N \tag{8.40}$$

Since $q = 0$, this means $v \in V_h$. Recall $\tilde{u}_h$ defined by

$$a(\tilde{u}_h - u, v) = 0, \qquad \forall v \in V_h.$$

Hence $\tilde{U}|_{I_n}$ is independent of $s \in I_n$, $k_n a(\tilde{U}, v) = \int_{I_n} a(u,v)ds = a(\int_{I_n}\tilde{u}_h(s)ds, v)$. So $\tilde{U}^n := \tilde{U}|_{I_n} = \frac{1}{k_n}\int_{I_n}\tilde{u}_h(s)ds$.

Let us compare $\tilde{U}^n$ and $u(t)$:

$$\|u(t_n)-\tilde{U}^n\| \le \left\|\frac{1}{k_n}\int_{I_n}\tilde{u}_h(s)-u(s)ds\right\| + \left\|\int_{I_n}|\dot{u}(s)|ds\right\|.$$

We have proved before (Theorem 7.1) that

$$\|\tilde{u}_h(s)-u(s)\| \le Ch\|\nabla(\tilde{u}_h(s)-u(s))\| \le Ch^2\|u(s)\|_{H^2(\Omega)}.$$

So, if we define $\theta = u - \tilde{U}$, then

$$\|\theta^n\| = \|u(t_n)-\tilde{U}^n\| \le h^2\max_{s\in I_n}\|u(s)\|_{H^2(\Omega)} + \int_{I_n}\|\dot{u}(s)\|ds. \tag{8.41}$$

So, we only need to compare U^n with $\tilde{U}^n$. The difference is now in W_{hk}.

II. Define $Z \in W_{hk}$ satisfying the discrete zero forcing backward heat equation with $Z^{N+1} = U^N - \tilde{U}^N$ as the initial input:

$$\begin{aligned} k_N a(w^N, Z^N) + (w^N, Z^N) &= (w^N, Z^{N+1}) \\ k_{N-1} a(w^{N-1}, Z^{N-1}) + (w^{N-1}, Z^{N-1}) &= (w^{N-1}, Z^N) \\ &\cdots\cdots\cdots \\ k_1 a(w^1, Z^1) + (w^1, Z^1) &= (w^1, Z^2) \end{aligned}$$

Summing up, we get

$$\sum_{n=1}^{N} k_n a(w^n, Z^n) + \sum_{n=2}^{N} (w^n - w^{n-1}, Z^n) + (w^1, Z^1) = (w^N, Z^{N+1}). \tag{8.42}$$

III. Let $w = U - \tilde{U} \in W_{hk}$ in (8.42). We get

$$\begin{aligned} \|U^N - \tilde{U}^N\|^2 =& \sum_{n=1}^{N} k_n a(U^n - \tilde{U}^n, Z^n) + \sum_{n=2}^{N} \Big((U^n - \tilde{U}^n) - (U^{n-1} - \tilde{U}^{n-1}), Z^n \Big) \\ &+ (U^1 - \tilde{U}^1, Z^1) \\ =& \sum_{n=1}^{N} \int_{I_n} a(u - \tilde{U}, Z) dt + \sum_{n=2}^{N} \Big((u^n - \tilde{U}^n) - (u^{n-1} - \tilde{U}^{n-1}), Z^n \Big) \\ &+ (u^1 - \tilde{U}^1, Z^1), \end{aligned} \tag{8.43}$$

where we have used

$$(U^n - U^{n-1}, Z^n) + k_n a(U^n, Z^n) = \int_{I_n} (f, Z^n) dt = (u^n - u^{n-1}, Z^n) + \int_{I_n} a(u, Z^n) dt$$

in the last step. Now, plugging (8.40) into (8.43), we get

$$\begin{aligned} \|U^N - \tilde{U}^N\|^2 &= \sum_{n=2}^{N} \Big((u^n - \tilde{U}^n) - (u^{n-1} - \tilde{U}^{n-1}), Z^n \Big) + (u^1 - \tilde{U}^1, Z^1) \\ &= \sum_{n=2}^{N} \big(\theta^n - \theta^{n-1}, Z^n \big) + (\theta^1, Z^1) \qquad (8.44) \\ &= (\theta^N, Z^N) - \sum_{n=1}^{N-1} (\theta^n, Z^{n+1} - Z^n). \qquad (8.45) \end{aligned}$$

We already have the estimation for θ from (8.41), we need to estimate Z. Similar to the proof of (8.36) and (8.39), we have

$$\|Z^N\| \leq \|Z^{N+1}\| = \|U^N - \tilde{U}^N\| \tag{8.46}$$

$$\sum_{n=1}^{N-1} \|Z^{n+1} - Z^n\| \leq C \left(1 + \log \frac{T}{k_N}\right)^{1/2} \|U^N - \tilde{U}^N\|. \tag{8.47}$$

So, plugging them into (8.45), and using

$$\sum (\theta^n, Z^{n+1} - Z^n) \leq \max_n \|\theta^n\| \left(\sum_n \|Z^{n+1} - Z^n\|\right),$$

we finally obtain

$$\|U^N - \tilde{U}^N\| \leq C \left(1 + \log \frac{T}{k_N}\right)^{1/2} \max_n \|\theta^n\|.$$

Together with (8.41), this finishes the proof. $\square$

9 Mixed finite element method and fluid dynamics

9.1 derivation of the equations

The conservation of mass leads to

$$\rho_t + \nabla \cdot (\rho \boldsymbol{u}) = 0.$$

The conservation of linear momentum leads to

$$(\rho \boldsymbol{u})_t + \nabla \cdot (\rho \boldsymbol{u} \otimes \boldsymbol{u}) + \nabla \cdot \sigma = \nabla \cdot \boldsymbol{\sigma} + \boldsymbol{f}.$$

The conservation of angular momentum (due to Euler) leads to

$$\boldsymbol{\sigma} = \boldsymbol{\sigma}^\tau.$$

Euler says

$$\boldsymbol{\sigma} = -pI.$$

Navier-Stokes says

$$\boldsymbol{\sigma} = \mu(\nabla \boldsymbol{u} + \nabla \boldsymbol{u}^\tau) + \lambda \nabla \cdot \boldsymbol{u} I - pI.$$

So we get the Navier-Stokes equation

$$\rho_t + \nabla \cdot (\rho \boldsymbol{u}) = 0, \tag{9.1}$$

$$\rho \boldsymbol{u}_t + \rho \boldsymbol{u} \cdot \nabla \boldsymbol{u} + \nabla p = \mu \Delta \boldsymbol{u} + (\mu + \lambda) \nabla \nabla \cdot \boldsymbol{u} + \boldsymbol{f}. \tag{9.2}$$

Now, we have three unknowns $(\rho, p, \boldsymbol{u}) \in \mathbb{R} \times \mathbb{R} \times \mathbb{R}^n$ with two equations. We need one more equation, this additional equation relates p to ρ and is called constitutive equation. To be more accurate, the constitutive equation relates p to ρ and temperature, then we need an additional equation for the temperature (the conservation of energy).

Without the ν and $\mu+\lambda$ terms in (9.2), the system is called Euler equation.

If we assume ρ=const and is known, and then rename p, $\boldsymbol{f}$ and then $\nu = \mu/\rho$, we can reduce one unknowns and get

$$\nabla \cdot \boldsymbol{u} = 0, \tag{9.3}$$

$$\boldsymbol{u}_t + \boldsymbol{u} \cdot \nabla \boldsymbol{u} + \nabla p = \nu \Delta \boldsymbol{u} + \boldsymbol{f}. \tag{9.4}$$

Now, you see that the term $\boldsymbol{u}\cdot\nabla\boldsymbol{u}$ is the inertial force and $\nu\Delta\boldsymbol{u}-\nabla p$ are the viscous force. If we assume the fluids are very viscous, we can neglect the inertial force term comparing with viscous term, and get the Stokes equations

$$\boldsymbol{u}_t+\nabla p-\nu\Delta\boldsymbol{u}=\boldsymbol{f} \qquad \text{in } \Omega, \qquad \boldsymbol{u}|_\Gamma=0 \tag{9.5}$$

$$\nabla\cdot\boldsymbol{u}=0 \qquad \text{in } \Omega \tag{9.6}$$

Now, we can consider an even simpler system: the steady state Stokes equations

$$\nabla p-\Delta\boldsymbol{u}=\boldsymbol{f} \qquad \text{in } \Omega, \qquad \boldsymbol{u}|_\Gamma=0 \tag{9.7}$$

$$\nabla\cdot\boldsymbol{u}=0 \qquad \text{in } \Omega. \tag{9.8}$$

Note that we have taken $\nu=1$, which is equivalent to divide ν on both sides of (9.5) and then rename $\boldsymbol{f}$ and p.

9.2 PDE theory in divergence free space

We are looking for $\boldsymbol{u}$ that are divergence free. So, we introduce the space

$$V=\{\boldsymbol{u}\in H_0^1(\Omega,\mathbb{R}^n),\nabla\cdot\boldsymbol{u}=0\}$$

and "ignore" (9.8) since we are looking for $\boldsymbol{u}$ in this space. Then, we dot (9.7) with test function in V and do integration to get

$$\int_\Omega \nabla p\cdot\boldsymbol{v}dx=-\int_\Omega p\nabla\cdot\boldsymbol{v}dx$$

which then drops due to $\nabla\cdot\boldsymbol{v}=0$. So, we conclude $\boldsymbol{u}\in V$ satisfies

$$(\nabla\boldsymbol{u},\nabla\boldsymbol{v})=(\boldsymbol{f},\boldsymbol{v}) \qquad \forall\boldsymbol{v}\in V. \tag{9.9}$$

Certainly the V is a linear space, and even more a Hilbert space. Lax-Milgram can be applied again to (9.9) to conclude the existence of the solution $\boldsymbol{u}\in V$.

We need the following theorems [see R.Teman "Navier-Stokes equations, theory and numerical analysis" Chap 1] to conclude the existence of pressure p.

Theorem 9.1 *Let Ω be an open set of $\mathbb{R}^n$ and $\boldsymbol{f} = (f_1, .., f_n)$, $f_i \in \mathcal{D}'$, $i = 1, ..., n$. A necessary and sufficient condition that*

$$\boldsymbol{f} = \nabla p$$

for some $p \in \mathcal{D}'$ is that

$$(\boldsymbol{f}, \boldsymbol{v}) = 0 \qquad \forall \boldsymbol{v} \in \mathcal{V} = \{\boldsymbol{v} \in \mathcal{D}(\Omega), \nabla \cdot \boldsymbol{v} = 0\}.$$

Theorem 9.2 *Let Ω be a bounded Lipschitz open set in $\mathbb{R}^n$. If a distribution p has all ites first-order derivatives in $H^{-1}(\Omega)$, then $p \in L^2(\Omega)$ and*

$$\|p\|_{L^2(\Omega)/\mathbb{R}} \leq C(\Omega)\|\nabla p\|_{H^{-1}(\Omega)}.$$

If Ω is any open set in $\mathbb{R}^n$, then $p \in L^2_{loc}(\Omega)$.

9.3 another weak form and inf-sup condition

However, it is hard to find a finite element space to approximate V due to the constraint $\nabla \cdot \boldsymbol{u} = 0$. The traditional approach is to use instead of (9.9), the weak form

$$(\nabla \boldsymbol{u}, \nabla \boldsymbol{v}) - (p, \nabla \cdot \boldsymbol{v}) = (\boldsymbol{f}, \boldsymbol{v}) \tag{9.10}$$

$$(q, \nabla \cdot \boldsymbol{u}) = 0 \tag{9.11}$$

for any $\boldsymbol{v} \in H^1_0(\Omega, \mathbb{R}^n)$ and $q \in L^2(\Omega)$ which are then in the unconstrained space. Equations (9.10)–(9.11) enable us to use the traditional finite element space to approximate $\boldsymbol{u}$ and p.

So, we are looking for $\boldsymbol{u}_h \in X_{0,h}$ and $p \in Y_h$ such that

$$(\nabla \boldsymbol{u}_h, \nabla \boldsymbol{v}_h) - (p_h, \nabla \cdot \boldsymbol{v}_h) = (\boldsymbol{f}, \boldsymbol{v}_h) \tag{9.12}$$

$$(q_h, \nabla \cdot \boldsymbol{u}_h) = 0 \tag{9.13}$$

for any $\boldsymbol{v}_h \in X_{0,h}$ and $q_h \in Y_h$. One of the first questions one should ask is if we have uniqueness of the solution, which is equivalent to ask if $\boldsymbol{f} = 0$, can one deduce $\boldsymbol{u}_h = 0$ and $p_h = 0$? $\boldsymbol{u}_h = 0$ can be easily derived after taking the test functions $\boldsymbol{v}_h = \boldsymbol{u}_h$ and $q_h = p_h$ and then adding up (9.12) and (9.13). One gets

$$(\nabla \boldsymbol{u}_h, \nabla \boldsymbol{u}_h) = 0$$

and hence $\boldsymbol{u}_h = 0$ if $\boldsymbol{f} = 0$.

The proof of $p_h = 0$ requires an extra compatibility condition between the finite element space $X_{0,h}$ and Y_h which basically says that $X_{0,h}$ must be large enough so that for any $q_h \in Y_h$, $q_h \neq$const, there is a $\boldsymbol{v}_h \in Y_h$ that can "feel" this nonconsant q_h in the sense that

$$(q_h, \nabla \cdot \boldsymbol{v}_h) \neq 0. \tag{9.14}$$

Assume (9.14) is true, let us now prove $p_h = 0$. From (9.12) and $\boldsymbol{u}_h = 0$, we get

$$(p_h, \nabla \cdot \boldsymbol{v}_h) = 0, \qquad \forall \boldsymbol{v}_h \in X_{0,h}. \tag{9.15}$$

But by assumption (9.14), if $p_h \neq$const, then there is a $\boldsymbol{v}_h \in Y_h$ such that

$$(p_h, \nabla \cdot \boldsymbol{v}_h) \neq 0$$

which contradict (9.15). So $p_h =$const.

To be able to prove full error estimates, we need a stronger condition than (9.14): there is a constant $\gamma > 0$ independent of h such that

$$\inf_{p_h \in Y_h, p_h \neq \text{const}} \sup_{\boldsymbol{v}_h \in X_{0,h}, \boldsymbol{v}_h \neq 0} \frac{(p_h, \nabla \cdot \boldsymbol{v}_h)}{\|p_h\|_{L^2} \|\boldsymbol{v}_h\|_{H^1}} \geq \gamma \tag{9.16}$$

Indeed, the truth is that from the PDE theory, One can manage to prove the following "continuous" inf-sup condition

$$\inf_{p \in L^2, p \neq \text{const}} \sup_{\boldsymbol{v} \in H_0^1, \boldsymbol{v} \neq 0} \frac{(p, \nabla \cdot \boldsymbol{v})}{\|p\|_{L^2} \|\boldsymbol{v}\|_{H^1}} \geq \gamma \tag{9.17}$$

which is used to establish the existence of solution in (9.10) and (9.11) .

9.4 error estimates

We can write the scheme in a more abstract form which can be applied to other situations where mixed approximation is used: we are looking for $\boldsymbol{u}_h \in X_h$ and $p \in Y_h$ such that

$$a(\boldsymbol{u}_h, \boldsymbol{v}_h) + b(\boldsymbol{v}_h, p_h) = f(\boldsymbol{v}_h) \tag{9.18}$$

$$b(\boldsymbol{u}_h, q_h) = g(q_h) \tag{9.19}$$

for any $\boldsymbol{v}_h \in X_h$ and $q_h \in Y_h$.

Theorem 9.3 *Consider (9.18)+(9.19). Assume that there exist constant $\alpha^* > 0$ and $\gamma^* > 0$ independent of h, such that*

(i) $a(\boldsymbol{v}_h, \boldsymbol{v}_h) \geq \alpha^* \|\boldsymbol{v}_h\|_X^2$

(ii) $\inf_{q_h \in Y_h, q_h \neq const} \sup_{\boldsymbol{v}_h \in X_{0,h}, \boldsymbol{v}_h \neq 0} \frac{b(\boldsymbol{v}_h, q_h)}{\|q_h\|_Y \|\boldsymbol{v}_h\|_X} \geq \gamma^*$

Then a discrete solution $(\boldsymbol{u}_h, p_h)$ exists and is unique. Moreover, if $(\boldsymbol{u}, p) \in X \times Y$ is the exact solution, then

$$\|\boldsymbol{u} - \boldsymbol{u}_h\|_X + \|p - p_h\|_Y \leq C \left(\inf_{\boldsymbol{v}_h \in X_h} \|\boldsymbol{u} - \boldsymbol{v}_h\|_X + \inf_{q_h \in Y_h} \|p - q_h\|_Y \right). \quad (9.20)$$

Proof: We will define a symmetric bilinear form and prove it satisfies the inf-sup condition. Then we prove (9.20) which is similar to what we did in the elliptic problem case.

(I) define

$$B[(\boldsymbol{u}, p); (\boldsymbol{v}, q)] = a(\boldsymbol{u}, \boldsymbol{v}) + b(\boldsymbol{u}, q) + b(\boldsymbol{v}, p)$$

which is a bilinear from on $(X \times Y) \times (X \times Y)$, and define the corresponding functional

$$F[(\boldsymbol{v}, q)] = f(\boldsymbol{v}) + g(q).$$

Then the problem is to find $(\boldsymbol{u}_h, p_h) \in X_h \times Y_h$ so that

$$B[(\boldsymbol{u}_h, p_h); (\boldsymbol{v}_h, q_h)] = F[(\boldsymbol{v}_h, q_h)], \qquad \forall (\boldsymbol{v}_h, q_h) \in X_h \times Y_h.$$

We need the following lemma whose proof will be given in the end:

Lemma 5 *Under assumption (i) and (ii), there is a $\gamma > 0$ independent of h so that*

$$\inf_{(\boldsymbol{w}_h, s_h)} \sup_{(\boldsymbol{v}_h, q_h)} \frac{B[(\boldsymbol{w}_h, s_h); (\boldsymbol{v}_h, q_h)]}{\|(\boldsymbol{w}_h, s_h)\| \|(\boldsymbol{v}_h, q_h)\|} \geq \gamma. \quad (9.21)$$

(II) We have the orthogonality property:

$$B[(\boldsymbol{u} - \boldsymbol{u}_h, p - p_h); (\boldsymbol{v}_h, q_h)] = 0, \qquad \forall (\boldsymbol{v}_h, q_h) \in X_h \times Y_h. \quad (9.22)$$

In the elliptic case, we then get $B[(\boldsymbol{u} - \boldsymbol{u}_h, p - p_h); (\boldsymbol{u} - \boldsymbol{u}_h, p - p_h)] = B[(\boldsymbol{u} - \boldsymbol{u}_h, p - p_h); (\boldsymbol{u} - \boldsymbol{v}_h, p - q_h)]$ and hence obtain the error estimates if

B is coercive. However, here we have to work in another way since we don't know if

$$B[(\boldsymbol{u}-\boldsymbol{u}_h, p-p_h); (\boldsymbol{u}-\boldsymbol{u}_h, p-p_h)] \geq c\|(\boldsymbol{u}-\boldsymbol{u}_h, p-p_h)\|^2.$$

What we know is that by (9.21), for any $(\boldsymbol{v}_h, q_h)$ (this *for any* is important and that is why we need an inf in (9.21), we put a max in (9.21) because we want a bigger γ):

$$\begin{aligned}\gamma\|(\boldsymbol{u}_h-\boldsymbol{v}_h); (p_h-q_h)\| &\leq \sup_{(\boldsymbol{w}_h, s_h)} \frac{B[(\boldsymbol{u}_h-\boldsymbol{v}_h, p_h-q_h); (\boldsymbol{w}_h, s_h)]}{\|(\boldsymbol{w}_h, s_h)\|} \\ &= \sup_{(\boldsymbol{w}_h, s_h)} \frac{B[(\boldsymbol{u}-\boldsymbol{v}_h, p-q_h); (\boldsymbol{w}_h, s_h)]}{\|(\boldsymbol{w}_h, s_h)\|}\end{aligned}$$

where we have used (9.22) in the last step. So,

$$\|(\boldsymbol{u}_h-\boldsymbol{v}_h, p_h-q_h)\| \leq \frac{\|B\|}{\gamma}\|(\boldsymbol{u}-\boldsymbol{v}_h, p-q_h)\|$$

for any $(\boldsymbol{v}_h, p_h) \in X_h \times Y_h$. Finally, we get

$$\|(\boldsymbol{u}-\boldsymbol{u}_h, p-p_h)\| \leq \|(\boldsymbol{u}-\boldsymbol{v}_h, p-q_h)\| + \|(\boldsymbol{u}_h-\boldsymbol{v}_h, p_h-q_h)\| \leq \left(1+\frac{\|B\|}{\gamma}\right)\|(\boldsymbol{u}-\boldsymbol{v}_h, p-q_h)\|$$

for any $(\boldsymbol{v}_h, p_h) \in X_h \times Y_h$. This proves (9.20). $\square$
Remark: If γ depends on h, say $\gamma = O(h)$, then the coefficient C in (9.20) will be of size $O(h^{-1})$ and the accuracy will degenerate.

Now, we turn to the proof of the lemma.
Proof of Lemma 5: We will prove (9.21) by showing that for any $(\boldsymbol{w}_h, s_h)$, there exists $(\boldsymbol{v}_h, q_h)$ such that

$$B[(\boldsymbol{w}_h, s_h); (\boldsymbol{v}_h, q_h)] \geq \gamma_1\|(\boldsymbol{w}_h, s_h)\|^2 \tag{9.23}$$

$$\gamma_2\|(\boldsymbol{w}_h, s_h)\| \geq \|(\boldsymbol{v}_h, q_h)\|. \tag{9.24}$$

By assumption (ii), for any $s_h \in Y_h$, there is a $\boldsymbol{z}_h \in X_h$ such that

$$\|\boldsymbol{z}_h\|_X = \|s_h\|_Y, \qquad -b(\boldsymbol{z}_h, s_h) \geq \gamma^*\|s_h\|_Y^2. \tag{9.25}$$

Now, for any given $(\boldsymbol{w}_h, s_h)$, take $\boldsymbol{v}_h = \boldsymbol{w}_h - \delta \boldsymbol{z}_h$ and $q_h = -s_h$ with the constant $\delta = \alpha^* \gamma^* / \|a\|^2$. Then

$$\begin{aligned}
&B[(\boldsymbol{w}_h, s_h); (\boldsymbol{v}_h, q_h)] = B[(\boldsymbol{w}_h, s_h); (\boldsymbol{w}_h - \delta \boldsymbol{z}_h, -s_h)] \\
=&a(\boldsymbol{w}_h, \boldsymbol{w}_h - \delta \boldsymbol{z}_h) + b(s_h, \boldsymbol{w}_h - \delta \boldsymbol{z}_h) + b(\boldsymbol{w}_h, -s_h) \\
=&a(\boldsymbol{w}_h, \boldsymbol{w}_h) - \delta a(\boldsymbol{w}_h, \boldsymbol{z}_h) - \delta b(\boldsymbol{z}_h, s_h) \\
\geq&\alpha^* \|\boldsymbol{w}_h\|_X^2 - \delta \|a\| \|\boldsymbol{w}_h\|_X \|s_h\|_Y + \delta \gamma^* \|s_h\|_Y^2 \\
\geq&\frac{\alpha^*}{2} \|\boldsymbol{w}_h\|_X^2 + \frac{\delta \gamma^*}{2} \|s_h\|_Y^2
\end{aligned}$$

since $\frac{\alpha^*}{2} \|\boldsymbol{w}_h\|_X^2 + \frac{\delta \gamma^*}{2} \|s_h\|_Y^2 \geq \delta \|a\| \|\boldsymbol{w}_h\|_X \|s_h\|_Y$ if $\delta = \alpha^* \gamma^* / \|a\|^2$. This proves (9.23).

$$\begin{aligned}
\|(\boldsymbol{v}_h, q_h)\| = \|\boldsymbol{v}_h\|_X + \|q_h\|_Y &\leq \|\boldsymbol{w}_h\|_X + \delta \|\boldsymbol{z}_h\|_X + \|s_h\|_Y \\
&\leq (1 + \delta) \left(\|\boldsymbol{w}_h\|_X + \|s_h\|_Y \right).
\end{aligned}$$

This proves (9.24). $\square$

10 ALBERTA

The following is what I remember what I did when installing ALBERTA.

(0) With cable plugged in and the required installation CDs being ready, install Fedora 6 with the default partition and layout configuration. In the first few steps, you will be asked for the IP address information etc. When I select what to be installed during the installation, I have chosen `mesa-libGL-...`, which I guess is the OpenGL we need later for `ALBERTA`.

After installation is finished, wait for a few minutes, then run the "Add/Remove Software" from "Applications". Search for (1) blas, (2) atlas, (3) sse2, (4) opengl, (5) DDD and selectively install the packages automatically detected by the system which you think might be related. Remark: I do not remember how I learned that and why I should install `atlas-sse2-....` `atlas` stands for automatic tuned linear algebra software and is the BLAS required in ALBERTA. I guess sse2 is for Pentium 4 CPU. My choice of sse2 might not be the right one, but it works. Indeed, I did not know where is the original `blas` being installed, except I guess `blas-sse2-...` is installed at `/usr/lib/sse2` . Searching for (4) opengl most likely is not necessary.

(1) Download ALBERTA-1.2 and also the free documentation for ALBERT-1.0 at `http://www.alberta-fem.de/`. Create a folder called, say, `alberta` to store the file. Unzip it and then read the README file.

(2) Download gltools-2.4 or higher at `http://www.wias-berlin.de/software/gltools/`. Unzip it, read the README.html. I do not understand why since I guess the default paths in the Makefile are not correct, but I do not need to change the Makefile when I install gltools. I do not have to do anything, but simply type in the `make lib bin` mentioned in the README.html.

(3) Download `GRAPE` at `http://www.iam.uni-bonn.de/sfb256/grape/ftp.html` and then register at `http://www.iam.uni-bonn.de/sfb256/grape/license.html` to obtain the associated .h file. In my case, I download `l_linux_ogl_glibc23_grape.h`. I have to save it as `grape.h` since `ALBERTA` will only recognize `grape.h`, not the other name. I create a folder called `GRAPE` inside the folder `alberta` to store the `.h` and `.a.gz`

files I got. I create a directory called `lib` inside `GRAPE` and moved the `.a.gz` file in it and then unzip it to `.a` file. I indeed do not remember how I unzip it since both `tar` and `gunzip` do not work. But I guess I can simply use "change name" to change `.a.gz` file to `.a` file. We do not need to unzip `.a` file to `.o` files. The `grape.h` file is put outside the directory `lib`. Later, after you use the `./configure`, read the `config.log` file to see if the correct information of `GRAPE` has been catched by `ALBERTA`.

(4) Install `ALBERTA`. After read the README and tried `./configure --help`, I issued the following command

```
./configure \
  --with-gltools-dir=/home/jieliu/alberta/gltools/gltools-2-4 \
  --with-gltools-lib=/home/jieliu/alberta/gltools/gltools-2-4 \
  --with-gltools-headers=/home/jieliu/alberta/gltools/gltools-2-4 \
  --with-grape-dir=/home/jieliu/alberta/GRAPE \
  --with-grape-lib=/home/jieliu/alberta/GRAPE/lib \
  --with-grape-headers=/home/jieliu/alberta/GRAPE \
  --with-blas-lib=/usr/lib/sse2
```

Finally ALBERTA will generate a `config.log` file and you can read it to see if you have installed all the packaged required. Now, run `make` follows by a `make install`. If you successfully installed `GRAPE`, you should find files like `alberta_movi22` in `alberta-1.2/GRAPE/mesh/2d`.

(5) Now, go to `alberta-1.2/DEMO/src/2d/` and then type in `make DEBUG=1 ellipt` or `make ellipt`, followed by a `./ellipt` to see the results and `DDD ellipt &` to start the debugging after calling `make DEBUG=1 ellipt`. You have to change the `ellipt.c` file in order for make to rebuild it. The source code `ellipt.c` is under folder `DEMO/src/Common`.

10.1 sample ALBERTA files

The following is the `ellipt.c` file in `alberta/alberta-1.2/DEMO/src/Common`

```
/*----------------------------------------------------------------------*/
/*  (c) by A. Schmidt and K.G. Siebert (1996-2004)                      */
/*----------------------------------------------------------------------*/

#include <alberta.h>

/*----------------------------------------------------------------------*/
/*  function for displaying mesh, discrete solution, and/or estimate    */
/*  defined in graphics.c                                               */
/*----------------------------------------------------------------------*/
void graphics(MESH *mesh, DOF_REAL_VEC *u_h, REAL (*get_est)(EL *el));

/*----------------------------------------------------------------------*/
/* global variables: finite element space, discrete solution            */
/*                   load vector and system matrix                      */
/*----------------------------------------------------------------------*/

static const FE_SPACE *fe_space;         /* initialized by init_dof_admin() */
static DOF_REAL_VEC   *u_h = nil;        /* initialized by build()          */
static DOF_REAL_VEC   *f_h = nil;        /* initialized by build()          */
static DOF_MATRIX     *matrix = nil;     /* initialized by build()          */

/*----------------------------------------------------------------------*/
/* init_dof_admin(): init DOFs for Lagrange elements of order           */
/*                   <polynomial degree>, called by GET_MESH()          */
/*----------------------------------------------------------------------*/

void init_dof_admin(MESH *mesh)
{
  FUNCNAME("init_dof_admin");
  int             degree = 1;
  const BAS_FCTS  *lagrange;

  GET_PARAMETER(1, "polynomial degree", "%d", &degree);
  lagrange = get_lagrange(degree);
  TEST_EXIT(lagrange)("no lagrange BAS_FCTS\n");
  fe_space = get_fe_space(mesh, lagrange->name, nil, lagrange);
  return;
}


/*----------------------------------------------------------------------*/
/* struct ellipt_leaf_data: structure for storing one REAL value on each    */
/*                          leaf element as LEAF_DATA                       */
/* init_leaf_data(): initialization of leaf data, called by GET_MESH()      */
/* rw_el_est():  return a pointer to the memory for storing the element     */
/*               estimate (stored as LEAF_DATA), called by ellipt_est()     */
/* get_el_est(): return the value of the element estimates (from LEAF_DATA),*/
/*               called by adapt_method_stat() and graphics()               */
/*----------------------------------------------------------------------*/

struct ellipt_leaf_data
{
```

```
  REAL estimate;            /*  one real for the estimate                  */
};

void init_leaf_data(LEAF_DATA_INFO *leaf_data_info)
{
  leaf_data_info->leaf_data_size = sizeof(struct ellipt_leaf_data);
  leaf_data_info->coarsen_leaf_data = nil; /* no transformation            */
  leaf_data_info->refine_leaf_data = nil;  /* no transformation            */
  return;
}

static REAL *rw_el_est(EL *el)
{
  if (IS_LEAF_EL(el))
    return(&((struct ellipt_leaf_data *)LEAF_DATA(el))->estimate);
  else
    return(nil);
}

static REAL get_el_est(EL *el)
{
  if (IS_LEAF_EL(el))
    return(((struct ellipt_leaf_data *)LEAF_DATA(el))->estimate);
  else
    return(0.0);
}

/*----------------------------------------------------------------------*/
/* For test purposes: exact solution and its gradient (optional)        */
/*----------------------------------------------------------------------*/

static REAL u(const REAL_D x)
{
  return(exp(-10.0*SCP_DOW(x,x)));
}

static const REAL *grd_u(const REAL_D x)
{
  static REAL_D grd;
  REAL          ux = exp(-10.0*SCP_DOW(x,x));
  int           n;

  for (n = 0;  n < DIM_OF_WORLD; n++)
    grd[n] = -20.0*x[n]*ux;

  return(grd);
}

/*----------------------------------------------------------------------*/
/* problem data: right hand side, boundary values                       */
/*----------------------------------------------------------------------*/

static REAL g(const REAL_D x)              /* boundary values, not optional */
{
  return(u(x));
}
```

```
static REAL f(const REAL_D x)              /* -Delta u, not optional       */
{
  REAL  r2 = SCP_DOW(x,x), ux  = exp(-10.0*r2);
  return(-(400.0*r2 - 20.0*DIM)*ux);
}


/*--------------------------------------------------------------------------*/
/* build(): assemblage of the linear system: matrix, load vector,           */
/*          boundary values, called by adapt_method_stat()                  */
/*          on the first call initialize u_h, f_h, matrix and information   */
/*          for assembling the system matrix                                */
/*                                                                          */
/* struct op_info: structure for passing information from init_element() to */
/*                 LALt()                                                   */
/* init_element(): initialization on the element; calculates the            */
/*                 coordinates and |det DF_S| used by LALt; passes these    */
/*                 values to LALt via user_data,                            */
/*                 called on each element by update_matrix()                */
/* LALt():         implementation of -Lambda id Lambda^t for -Delta u,      */
/*                 called by update_matrix() after init_element()           */
/*--------------------------------------------------------------------------*/

struct op_info
{
  REAL_D  Lambda[DIM+1];    /*  the gradient of the barycentric coordinates */
  REAL    det;              /*  |det D F_S|                                 */
};

static void init_element(const EL_INFO *el_info, const QUAD *quad[3], void *ud)
{
  struct op_info *info = ud;

  info->det = el_grd_lambda(el_info, info->Lambda);
  return;
}

const REAL (*LALt(const EL_INFO *el_info, const QUAD *quad,
          int iq, void *ud))[DIM+1]
{
  struct op_info *info = ud;
  int            i, j, k;
  static REAL    LALt[DIM+1][DIM+1];

  for (i = 0; i <= DIM; i++)
    for (j = i; j <= DIM; j++)
    {
      for (LALt[i][j] = k = 0; k < DIM_OF_WORLD; k++)
    LALt[i][j] += info->Lambda[i][k]*info->Lambda[j][k];
      LALt[i][j] *= info->det;
      LALt[j][i] = LALt[i][j];
    }

  return((const REAL (*)[DIM+1]) LALt);
}

static void build(MESH *mesh, U_CHAR flag)
```

```
{
  FUNCNAME("build");
  static const EL_MATRIX_INFO *matrix_info = nil;
  const QUAD                  *quad;

  dof_compress(mesh);
  MSG("%d DOFs for %s\n", fe_space->admin->size_used, fe_space->name);

  if (!u_h)                 /*  access matrix and vector for linear system */
  {
    matrix = get_dof_matrix("A", fe_space);
    f_h    = get_dof_real_vec("f_h", fe_space);
    u_h    = get_dof_real_vec("u_h", fe_space);
    u_h->refine_interpol = fe_space->bas_fcts->real_refine_inter;
    u_h->coarse_restrict = fe_space->bas_fcts->real_coarse_inter;
    dof_set(0.0, u_h);      /*  initialize u_h  !                          */
  }

  if (!matrix_info)           /* information for matrix assembling         */
  {
    OPERATOR_INFO  o_info = {nil};

    o_info.row_fe_space   = o_info.col_fe_space = fe_space;
    o_info.init_element   = init_element;
    o_info.LALt           = LALt;
    o_info.LALt_pw_const  = true;        /* pw const. assemblage is faster */
    o_info.LALt_symmetric = true;        /* symmetric assemblage is faster */
    o_info.use_get_bound  = true;        /* Dirichlet boundary conditions! */
    o_info.user_data = MEM_ALLOC(1, struct op_info);         /* user data! */
    o_info.fill_flag = CALL_LEAF_EL|FILL_COORDS;

    matrix_info = fill_matrix_info(&o_info, nil);
  }

  clear_dof_matrix(matrix);                /* assembling of matrix         */
  update_matrix(matrix, matrix_info);

  dof_set(0.0, f_h);                       /* assembling of load vector    */
  quad = get_quadrature(DIM, 2*fe_space->bas_fcts->degree - 2);
  L2scp_fct_bas(f, quad, f_h);

  dirichlet_bound(g, f_h, u_h, nil);           /*  boundary values         */
  return;
}


/*--------------------------------------------------------------------------*/
/* solve(): solve the linear system, called by adapt_method_stat()          */
/*--------------------------------------------------------------------------*/

static void solve(MESH *mesh)
{
  FUNCNAME("solve");
  static REAL       tol = 1.e-8;
  static int        miter = 1000, info = 2, icon = 1, restart = 0;
  static OEM_SOLVER solver = NoSolver;
```

```
  if (solver == NoSolver)
  {
    tol = 1.e-8;
    GET_PARAMETER(1, "solver", "%d", &solver);
    GET_PARAMETER(1, "solver tolerance", "%f", &tol);
    GET_PARAMETER(1, "solver precon", "%d", &icon);
    GET_PARAMETER(1, "solver max iteration", "%d", &miter);
    GET_PARAMETER(1, "solver info", "%d", &info);
    if (solver == GMRes)
      GET_PARAMETER(1, "solver restart", "%d", &restart);
  }
  oem_solve_s(matrix, f_h, u_h, solver, tol, icon, restart, miter, info);

  graphics(mesh, u_h, nil);
  return;
}

/*----------------------------------------------------------------------*/
/* Functions for error estimate:                                        */
/* estimate():   calculates error estimate via ellipt_est()             */
/*               calculates exact error also (only for test purpose),   */
/*               called by adapt_method_stat()                          */
/* r():          calculates the lower order terms of the element residual */
/*               on each element at the quadrature node iq of quad      */
/*               argument to ellipt_est() and called by ellipt_est()    */
/*----------------------------------------------------------------------*/

static REAL r(const EL_INFO *el_info, const QUAD *quad, int iq, REAL uh_iq,
              const REAL_D grd_uh_iq)
{
  REAL_D      x;
  coord_to_world(el_info, quad->lambda[iq], x);
  return(-f(x));
}

#define EOC(e,eo) log(eo/MAX(e,1.0e-15))/M_LN2

static REAL estimate(MESH *mesh, ADAPT_STAT *adapt)
{
  FUNCNAME("estimate");
  static int     degree, norm = -1;
  static REAL    C[3] = {1.0, 1.0, 0.0};
  static REAL    est, est_old = -1.0, err, err_old = -1.0;
  static FLAGS r_flag = 0;  /* = (INIT_UH | INIT_GRD_UH),  if needed by r() */
  REAL_DD        A = {{0.0}};
  int            n;
  const QUAD     *quad;

  for (n = 0; n < DIM_OF_WORLD; n++)
    A[n][n] = 1.0;   /* set diogonal of A; all other elements are zero      */

  if (norm < 0)
  {
    norm = H1_NORM;
    GET_PARAMETER(1, "error norm", "%d", &norm);
    GET_PARAMETER(1, "estimator C0", "%f", &C[0]);
    GET_PARAMETER(1, "estimator C1", "%f", &C[1]);
```

```
    GET_PARAMETER(1, "estimator C2", "%f", &C[2]);
  }
  degree = 2*u_h->fe_space->bas_fcts->degree;
  est = ellipt_est(u_h, adapt, rw_el_est, nil, degree, norm, C,
           (const REAL_D *) A, r, r_flag);

  MSG("estimate   = %.8le", est);
  if (est_old >= 0)
    print_msg(", EOC: %.2lf\n", EOC(est,est_old));
  else
    print_msg("\n");
  est_old = est;

  quad = get_quadrature(DIM, degree);
  if (norm == L2_NORM)
    err = L2_err(u, u_h, quad, 0, nil, nil);
  else
    err = H1_err(grd_u, u_h, quad, 0, nil, nil);

  MSG("||u-uh||%s = %.8le", norm == L2_NORM ? "L2" : "H1", err);
  if (err_old >= 0)
    print_msg(", EOC: %.2lf\n", EOC(err,err_old));
  else
    print_msg("\n");
  err_old = err;
  MSG("||u-uh||%s/estimate = %.2lf\n", norm == L2_NORM ? "L2" : "H1",
      err/MAX(est,1.e-15));

  graphics(mesh, nil, get_el_est);
  return(adapt->err_sum);
}

/*--------------------------------------------------------------------------*/
/* main program                                                             */
/*--------------------------------------------------------------------------*/

int main(int argc, char **argv)
{
  FUNCNAME("main");
  MESH   *mesh;
  int    n_refine = 0;
  static ADAPT_STAT *adapt;
  char   filename[100];

/*--------------------------------------------------------------------------*/
/*  first of all, init parameters of the init file                          */
/*--------------------------------------------------------------------------*/

  init_parameters(0, "INIT/ellipt.dat");

/*--------------------------------------------------------------------------*/
/*  get a mesh, and read the macro triangulation from file                  */
/*--------------------------------------------------------------------------*/

  mesh = GET_MESH("ALBERTA mesh", init_dof_admin, init_leaf_data);
  GET_PARAMETER(1, "macro file name", "%s", filename);
  read_macro(mesh, filename, nil);
```

```
  GET_PARAMETER(1, "global refinements", "%d", &n_refine);
  global_refine(mesh, n_refine*DIM);
  graphics(mesh, nil, nil);

/*--------------------------------------------------------------------------*/
/*  init adapt structure and start adaptive method                          */
/*--------------------------------------------------------------------------*/

  adapt = get_adapt_stat("ellipt", "adapt", 2, nil);
  adapt->estimate = estimate;
  adapt->get_el_est = get_el_est;
  adapt->build_after_coarsen = build;
  adapt->solve = solve;

  adapt_method_stat(mesh, adapt);
  WAIT_REALLY;
  return(0);
}
```

The following is the function `adapt_method_stat()` defined in adapt.c file in `alberta/alberta-1.2/ALBERTA/src/Common`

```
/*--------------------------------------------------------------------------*/
/* adapt_method_stat: adapt for stationary problems                         */
/*--------------------------------------------------------------------------*/


void adapt_method_stat(MESH *mesh, ADAPT_STAT *adapt)
{
  FUNCNAME("adapt_method_stat");
  int      iter;
  REAL     est;
  clock_t  first;

  TEST_EXIT(mesh)("no MESH\n");
  TEST_EXIT(adapt)("no ADAPT_STAT\n");

  /* get solution on initial mesh */
  if (adapt->build_before_refine)  adapt->build_before_refine(mesh, 0);
  if (adapt->build_before_coarsen) adapt->build_before_coarsen(mesh, 0);
  if (adapt->build_after_coarsen)  adapt->build_after_coarsen(mesh, 0);
  if (adapt->solve)
  {
    first = clock();
    adapt->solve(mesh);
    INFO(adapt->info,8)("solution of discrete system needed %.5lg seconds\n",
            TIME_USED(first,clock()));
  }

  first = clock();
  est = adapt->estimate ? adapt->estimate(mesh, adapt) : 0.0;
  INFO(adapt->info,8)("estimation of the error needed %.5lg seconds\n",
              TIME_USED(first,clock()));

  for (iter = 0;
       (est > adapt->tolerance) &&
```

```
     ((adapt->max_iteration <= 0) || (iter < adapt->max_iteration));
       iter++)
  {
    if (adapt_mesh(mesh, adapt))
    {
      if (adapt->solve)
      {
    first = clock();
    adapt->solve(mesh);
    INFO(adapt->info,8)
      ("solution of discrete system needed %.5lg seconds\n",
       TIME_USED(first,clock()));
      }
      first = clock();
      est = adapt->estimate ? adapt->estimate(mesh, adapt) : 0.0;
      INFO(adapt->info,8)("estimation of the error needed %.5lg seconds\n",
              TIME_USED(first,clock()));
    }
    else
    {
      ERROR("no mesh adaption, but estimate above tolerance ???\n");
      break;
    }
    INFO(adapt->info, 4)("iter: %d", iter+1);
    PRINT_INFO(adapt->info, 4)(", tol = %.4le", adapt->tolerance);
    PRINT_INFO(adapt->info, 4)(", estimate = %.4le\n", est);
  }

  if ((iter > adapt->max_iteration) && (adapt->max_iteration > 0))
  {
    MSG("max_iterations REACHED: %d\n", adapt->max_iteration);
    MSG("prescribed tolerance    %le\n", adapt->tolerance);
    MSG("finished with estimate  %le\n", est);
  }
  else
  {
    INFO(adapt->info, 2)("no of iterations:       %d\n", iter);
    INFO(adapt->info, 2)("prescribed tolerance    %.4le\n", adapt->tolerance);
    INFO(adapt->info, 2)("finished with estimate  %.4le\n", est);
  }
  return;
}
```

10.2 example of solving Poisson equation without adaptation

The following is the `uniform_refine_ellipt.c` which does not call `adapt_method_stat()`. I am not sure if it is the best way to implement it. You may have your own way to do it under ALBERTA.

```
/*--------------------------------------------------------------------------*/
/* ALBERTA:                                                                 */
/* description:  solver for an elliptic model problem                       */
/*                        -\Delta u = f  in \Omega                          */
```

```
/*                               u = g  on \partial \Omega                */
/*--------------------------------------------------------------------------*/
/*  authors:   Alfred Schmidt                                               */
/*             Kunibert G. Siebert                                          */
/*--------------------------------------------------------------------------*/
/*  (c) by A. Schmidt and K.G. Siebert (1996-2004)                          */
/*--------------------------------------------------------------------------*/
/*  modified by Jie Liu for a much simpler uniform refine case              */

#include <alberta.h>

/*--------------------------------------------------------------------------*/
/*  function for displaying mesh, discrete solution, and/or estimate        */
/*  defined in graphics.c                                                   */
/*--------------------------------------------------------------------------*/
void graphics(MESH *mesh, DOF_REAL_VEC *u_h, REAL (*get_est)(EL *el));

/*--------------------------------------------------------------------------*/
/* global variables: finite element space, discrete solution                */
/*                   load vector and system matrix                          */
/*--------------------------------------------------------------------------*/

static const FE_SPACE *fe_space;         /* initialized by init_dof_admin() */
static DOF_REAL_VEC   *u_h = nil;        /* initialized by build()          */
static DOF_REAL_VEC   *f_h = nil;        /* initialized by build()          */
static DOF_MATRIX     *matrix = nil;     /* initialized by build()          */

/*--------------------------------------------------------------------------*/
/* init_dof_admin(): init DOFs for Lagrange elements of order               */
/*                   <polynomial degree>, called by GET_MESH()              */
/*--------------------------------------------------------------------------*/

void init_dof_admin(MESH *mesh)
{
  FUNCNAME("init_dof_admin");
  int             degree = 1;
  const BAS_FCTS  *lagrange;

  GET_PARAMETER(1, "polynomial degree", "%d", &degree);
  lagrange = get_lagrange(degree);
  TEST_EXIT(lagrange)("no lagrange BAS_FCTS\n");
  fe_space = get_fe_space(mesh, lagrange->name, nil, lagrange);
  return;
}

/*--------------------------------------------------------------------------*/
/* For test purposes: exact solution and its gradient (optional)            */
/*--------------------------------------------------------------------------*/

static REAL u(const REAL_D x)
{
  return(exp(-10.0*SCP_DOW(x,x)));
}

static const REAL *grd_u(const REAL_D x)
{
  static REAL_D grd;
```

```
  REAL          ux = exp(-10.0*SCP_DOW(x,x));
  int           n;

  for (n = 0;  n < DIM_OF_WORLD; n++)
    grd[n] = -20.0*x[n]*ux;

  return(grd);
}

/*---------------------------------------------------------------------------*/
/* problem data: right hand side, boundary values                            */
/*---------------------------------------------------------------------------*/

static REAL g(const REAL_D x)              /* boundary values, not optional */
{
  return(u(x));
}

static REAL f(const REAL_D x)              /* -Delta u, not optional        */
{
  REAL  r2 = SCP_DOW(x,x), ux  = exp(-10.0*r2);
  return(-(400.0*r2 - 20.0*DIM)*ux);
}

/*---------------------------------------------------------------------------*/
/* build(): assemblage of the linear system: matrix, load vector,            */
/*          boundary values, called by adapt_method_stat()                   */
/*          on the first call initialize u_h, f_h, matrix and information    */
/*          for assembling the system matrix                                 */
/*                                                                           */
/* struct op_info: structure for passing information from init_element() to */
/*                 LALt()                                                    */
/* init_element(): initialization on the element; calculates the             */
/*                 coordinates and |det DF_S| used by LALt; passes these     */
/*                 values to LALt via user_data,                             */
/*                 called on each element by update_matrix()                 */
/* LALt():         implementation of -Lambda id Lambda^t for -Delta u,       */
/*                 called by update_matrix() after init_element()            */
/*---------------------------------------------------------------------------*/

struct op_info
{
  REAL_D  Lambda[DIM+1];    /*  the gradient of the barycentric coordinates */
  REAL    det;              /*  |det D F_S|                                 */
};

static void init_element(const EL_INFO *el_info, const QUAD *quad[3], void *ud)
{
  struct op_info *info = ud;

  info->det = el_grd_lambda(el_info, info->Lambda);
  return;
}

const REAL (*LALt(const EL_INFO *el_info, const QUAD *quad,
          int iq, void *ud))[DIM+1]
{
```

```
  struct op_info *info = ud;
  int            i, j, k;
  static REAL    LALt[DIM+1][DIM+1];

  for (i = 0; i <= DIM; i++)
    for (j = i; j <= DIM; j++)
    {
      for (LALt[i][j] = k = 0; k < DIM_OF_WORLD; k++)
    LALt[i][j] += info->Lambda[i][k]*info->Lambda[j][k];
      LALt[i][j] *= info->det;
      LALt[j][i] = LALt[i][j];
    }

  return((const REAL (*)[DIM+1]) LALt);
}

static void build(MESH *mesh, U_CHAR flag)
{
  FUNCNAME("build");
  static const EL_MATRIX_INFO *matrix_info = nil;
  const QUAD                  *quad;

  dof_compress(mesh);
  MSG("%d elements, %d DOFs for %s\n", mesh->n_elements, \
      fe_space->admin->size_used, fe_space->name);

  if (!u_h)                 /*  access matrix and vector for linear system */
  {
    matrix = get_dof_matrix("A", fe_space);
    f_h    = get_dof_real_vec("f_h", fe_space);
    u_h    = get_dof_real_vec("u_h", fe_space);
    u_h->refine_interpol = nil;
    u_h->coarse_restrict = nil;
    dof_set(0.0, u_h);      /*  initialize u_h  !                          */
  }

  if (!matrix_info)           /* information for matrix assembling         */
  {
    OPERATOR_INFO  o_info = {nil};

    o_info.row_fe_space   = o_info.col_fe_space = fe_space;
    o_info.init_element   = init_element;
    o_info.LALt           = LALt;
    o_info.LALt_pw_const  = true;        /* pw const. assemblage is faster */
    o_info.LALt_symmetric = true;        /* symmetric assemblage is faster */
    o_info.use_get_bound  = true;        /* Dirichlet boundary conditions! */
    o_info.user_data = MEM_ALLOC(1, struct op_info);         /* user data! */
    o_info.fill_flag = CALL_LEAF_EL|FILL_COORDS;

    matrix_info = fill_matrix_info(&o_info, nil);
  }

  clear_dof_matrix(matrix);                /* assembling of matrix         */
  update_matrix(matrix, matrix_info);

  dof_set(0.0, f_h);                       /* assembling of load vector    */
  quad = get_quadrature(DIM, 2*fe_space->bas_fcts->degree - 2);
```

```
  L2scp_fct_bas(f, quad, f_h);

  dirichlet_bound(g, f_h, u_h, nil);           /*  boundary values         */
  return;
}


/*--------------------------------------------------------------------------*/
/* main program                                                             */
/*--------------------------------------------------------------------------*/
int main(int argc, char **argv)
{
  FUNCNAME("main");
  MESH   *mesh;
  char   filename[100];
  int    n_init_refine = 0, n_repeat_refine=0, i;
  static REAL       tol = 1.e-8;
  static int        miter = 1000, info = 2, icon = 1, restart = 0;
  static OEM_SOLVER solver = NoSolver;
  double *u_L2_err, *u_H1_err;

/*--------------------------------------------------------------------------*/
/*  first of all, init parameters of the init file                          */
/*--------------------------------------------------------------------------*/
  init_parameters(0, "INIT/uniform_refine_ellipt.dat");

/*--------------------------------------------------------------------------*/
/*  get a mesh, and read the macro triangulation from file                  */
/*--------------------------------------------------------------------------*/
  mesh = GET_MESH("ALBERTA mesh", init_dof_admin, nil);
  // GET_MESH will call init_dof_admin which initializes the global variable fe_space
  mesh->preserve_coarse_dofs = 0;  // since we only want to check the accuracy

  GET_PARAMETER(1, "macro file name", "%s", filename);
  read_macro(mesh, filename, nil);
  GET_PARAMETER(1, "global init refinements", "%d", &n_init_refine);
  GET_PARAMETER(1, "global repeat refinements", "%d", &n_repeat_refine);
  global_refine(mesh, n_init_refine*DIM);

/*--------------------------------------------------------------------------*/
/*  setup the parameters for the global variable "solver"                   */
/*--------------------------------------------------------------------------*/
  if (solver == NoSolver)
  {
    tol = 1.e-8;
    GET_PARAMETER(1, "solver", "%d", &solver);
    GET_PARAMETER(1, "solver tolerance", "%f", &tol);
    GET_PARAMETER(1, "solver precon", "%d", &icon);
    GET_PARAMETER(1, "solver max iteration", "%d", &miter);
    GET_PARAMETER(1, "solver info", "%d", &info);
    if (solver == GMRes)
      GET_PARAMETER(1, "solver restart", "%d", &restart);
  }

/*--------------------------------------------------------------------------*/
/* start the accuracy check:                                                */
/*--------------------------------------------------------------------------*/
```

```
  u_L2_err = (double *)malloc(sizeof(double)*n_repeat_refine);
  u_H1_err = (double *)malloc(sizeof(double)*n_repeat_refine);

  for (i=0; i<n_repeat_refine; i++){
      global_refine(mesh, DIM);
      // global_refine will call function refine(mesh), which automatically resize
      // the u_h, f_h if u_h, f_h already exist. So the mesh have to be able to
      // "recognize" u_h and f_h. How each variable know each other?
      // For example, u_h and f_h are generated with
      // u_f = get_dof_real_vec("u_h", fe_space); // see build()
      // in this way, u_h is recognized by fe_space->admin which point to a DOF_ADMIN
      // variable. mesh also has a pointer point to the same variable,
      // therefore mesh is able to find and take care of u_h when it is being refined.
      // Indeed, the truth might be more complicated:
      // In DDD, by setting the watch point of u_h->size, we can see u_h change
      // size by calling MEM_REALLOC() in the function enlarge_dof_list(). The function
      // enlarge_dof_list() is called by get_dof_index(), which is called by get_dof(),
      // which is called by bisect_patch(), which is called by refine_function(),
      // which are called by refine(), which are called by global_refine().

      build(mesh, 0);
      // allocate memory for the global variables matrix, u_h and f_h if they are still nil
      //
      // build also setup the Dirichlet boundary values by call Dirichlet_bound

      oem_solve_s(matrix, f_h, u_h, solver, tol, icon, restart, miter, info);
      u_L2_err[i] = L2_err(u,    u_h,nil,0,nil,nil);
      u_H1_err[i] = H1_err(grd_u,u_h,nil,0,nil,nil);
      printf("u_L2_err = %f, u_H1_err = %f \n", u_L2_err[i], u_H1_err[i]);
      graphics(mesh, u_h, nil);
  }

  for (i=0; i<n_repeat_refine-1; i++){
      printf("L2_err[%d]/L2_err[%d] = %f, H1_err[%d]/H1_err[%d] = %f \n", \
              i, i+1, u_L2_err[i]/u_L2_err[i+1], i, i+1, u_H1_err[i]/u_H1_err[i+1]);
  }

  WAIT_REALLY;
  return(0);
}
```

I also need to write the following `uniform_refine_ellipt.dat` file and put it under `alberta-1.2/DEMO/src/2d/INIT`

```
macro file name:      Macro/macro.amc
global init refinements:   1
global repeat refinements:  4
polynomial degree:    2

% graphic windows: solution, estimate, and mesh if size > 0
graphic windows:      300 0 0

graphic range:  0.0 -1.0

solver:                2 % 1: BICGSTAB 2: CG 3: GMRES 4: ODIR 5: ORES
solver max iteration:  1000
solver restart:        10  %  only used for GMRES
```

```
solver tolerance:      1.e-8
solver info:           2
solver precon:         2   % 0: no precon 1: diag precon
                           % 2: HB precon 3: BPX precon

error norm:           2   % 1: H1_NORM, 2: L2_NORM

WAIT: 1
```

Finally, I need to add the following to the Makefile in `alberta-1.2/DEMO/src/2d`. Then I can run `make uniform_refine_ellipt` to generate the `.o` file to run.

```
UNIFORM_REFINE_ELLIPT_OFILES = uniform_refine_ellipt.o graphics.o

uniform_refine_ellipt: $(UNIFORM_REFINE_ELLIPT_OFILES)
    $(LINK) $(UNIFORM_REFINE_ELLIPT_OFILES) $(LIBS)
```

11 Homework and Projects

11.1 Homework

Suppose you use $P1$ finite element. Prove both L^2 and H^1 error estimates for

$$-\varepsilon\Delta u + u_x + u_y = f \qquad \text{in } \Omega = [0,1]\times[0,1], \qquad u = 0 \quad \text{on } \Gamma$$

for $\varepsilon = 0.1$. If you are math major, clearly state the regularity you need for f, otherwise, you can assume u is $C^\infty(\bar{\Omega})$. (Hint for math major: in the book "finite element methods for Navier-Stokes equations" by V. Girault and P.-A. Raviart, page 18, it cites the results of Grisvard that if Ω is a bounded polygon with no reentrant corner, if $f \in L^2(\Omega)$, then $u \in H^2(\Omega)$.)

11.2 Project

Solve

$$-\varepsilon\Delta u + u_x + u_y = f \qquad \text{in } \Omega = [0,1]\times[0,1], \qquad u = g \quad \text{on } \Gamma$$

for $\varepsilon = 0.1$. Suppose the exact solution is $u_{ex} = \cos(2\pi x)\sin(2\pi y)$: Use $P1$ and then $P2$ finite elements. In both case, print out $\|u_h - u_{ex}\|_{L^2}$ and $\|\nabla(u_h - u_{ex})\|_{L^2}$ for different h and study the order of accuracy.

Hint: several issues that are easy to make mistake: (1) change the linear solver from CG to GMRES since the system now is not symmetric. (2) pay attention to the negative sign in $-\varepsilon\Delta$ when compute f. (3) after write the `Lb0` function, add the following to the function `build`

```
o_info.Lb0            = Lb0;
o_info.Lb0_pw_const   = true;
```

The function `Lb0` can be as follows

```
const REAL *Lb0(const EL_INFO *el_info, const QUAD *quad, int iq, void *ud)
{ struct op_info *info = ud;
  int            i, j;
  static REAL    Lb[DIM+1];
  for (i = 0; i <= DIM; i++){
      for (Lb[i]=j=0; j < DIM; j++)
          Lb[i] += info->Lambda[i][j];
      Lb[i]*=info->det;
  }
  return Lb;
}
```

12 Extra reading: solving parabolic equation with finite difference

Consider $\partial_t u = \nu \partial_x^2 u$, $u(x,0) = f(x)$ and $u(0,t) = g_1(t)$ and $u(1,t) = g_2(t)$. We will show how to solve this equation with finite difference method, which should be compared with what we have already learned in this course.

12.1 method of line

$$\frac{dU}{dt} = \frac{\nu}{h^2} AU + \frac{\nu}{h^2} F, \qquad U(0) = U_0. \tag{12.26}$$

where $A = \text{tridiag}(1, -2, 1)$ and $F = (g_1(t), 0, ..., 0, g_2(t))^\tau$. $h = 1/M$ and A is a $(M-1) \times (M-1)$ matrix. Recall that the eigenvalues of A are $\lambda_j = (2\cos\theta_j - 2) < 0$ with $\theta_j = \frac{j\pi}{M}$, $j = 1, ..., M-1$. The associated eigenvectors are $(\sin\theta_j, \sin(2\theta_j), ...)$. Note that this is just $\sin j\pi x$. Among the eigenvalues, the one with largest absolute value is $\lambda_{\max} = 2\cos((M-1)\pi/M) - 2 \approx -4$ and the one with smallest absolute value is $\lambda_{\min} = 2\cos(\pi/M) - 2 \approx -\pi^2/M^2$ when M large.

So the eigenvalues for $\frac{\nu}{h^2}A$ goes from $-\pi^2\nu$ all the way up (in negative direction) to $-4\nu M^2$ which goes to $-\infty$ as M goes to infinity.

Suppose we use forward Euler,

$$\frac{U^{n+1} - U^n}{k} = \frac{\nu}{h^2} AU^n + \frac{\nu}{h^2} F^n. \tag{12.27}$$

Recall that when we apply forward Euler to $y' = \lambda y$ with $\Re(\lambda) < 0$, we get $y^{n+1} = (1 + \lambda k) y^n$. To get stability, we must have $|1 + \lambda k| \le 1$.

So, for the current case, for stability, we must have $-2 \le \frac{\nu}{h^2}\lambda_j k \le 0$. Note that when $\lambda = \lambda_{\min}$, we can take large k. What really enforce us to take a small k is when $\lambda = \lambda_{\max}$. Associated with $\lambda_{\max}$ is the eigenvector associated with high frequency. But high frequency decays very fast for the exact PDE. That's why this is so bad — even high frequency disappears very soon in the PDE, we have to take large time step all the way down to the end in numerics!

The step restriction is therefore $k \le \frac{h^2}{2\nu}$ for using forward Euler.

This simply means the ODE system (12.26) is very stiff. The way to overcome it is to use the following two method. There are certainly other choices and that's why ODE solver is so fascinating.

If we use backward Euler on the test problem $y' = \lambda y$, we have $y^{n+1} = \frac{1}{1-\lambda k} y^n$. We can take any k. If using Crank-Nicolson,

$$\frac{y^{n+1} - y^n}{k} = \frac{1}{2}\lambda(y^{n+1} + y^n).$$

$y^{n+1} = \frac{1+\frac{1}{2}\lambda k}{1-\frac{1}{2}\lambda k} y^n$. Again, we can take any k. So, it is unconditionally stable for the later two cases.

12.2 local truncation error

First, the exact solution satisfies

$$\frac{d\vec{u}}{dt} = \frac{\nu}{h^2} A\vec{u} + \frac{\nu}{h^2} F + O(h^2)$$

at any t where $\vec{u} = \vec{u}(t) = (u(x_1, t), ..., u(x_{M-1}, t))^\tau$. Then,

$$\begin{aligned}
&\frac{\vec{u}(t_{n+1}) - \vec{u}(t_n)}{k} \\
=&\frac{d\vec{u}(t_{n+1/2})}{dt} + O(k^2) = \frac{\nu}{h^2} A\vec{u}(t_{n+1/2}) + \frac{\nu}{h^2} F(t_{n+1/2}) + O(h^2 + k^2) \\
=&\frac{1}{2}\frac{\nu}{h^2} A(\vec{u}(t_{n+1}) + \vec{u}(t_n)) + \frac{1}{2}\frac{\nu}{h^2}(F(t_{n+1}) + F(t_n)) + O(h^2 + k^2)
\end{aligned}$$

12.3 von Neumann analysis: stability and accuracy

In order to apply the von Neumann analysis, we assume periodic boundary condition or in a infinite domain. Recall that in matlab, the FFT for a length N vector is defined by (Try `help fft` in matlab. The index starts form 1.)

$$\hat{v}(k) = \sum_{j=1}^{N} v(j) e^{-2\pi i(k-1)(j-1)/N}, \qquad v(j) = \frac{1}{N}\sum_{k=1}^{N} \hat{v}(k) e^{2\pi i(k-1)(j-1)/N}.$$

We make the ansatz

$$u_j^n = \hat{u}^n(\omega) e^{2\pi i \omega x_j}, \qquad \text{for } j = 0, 1, ..., M \tag{12.28}$$

In order for it to satisfies the periodic boundary condition, we must have $\omega \in \mathbb{Z}$. Note that for a fixed ω, $\vec{r}_\omega = (e^{2\pi i \omega x_0}, ..., e^{2\pi i \omega x_M})$ is the eigenfunction for

the difference operator associated with ∂_x^2 (with periodic boundary condition) as we will see in a moment (see (12.30)). In fact, we will see that $\vec{r}_{-\omega}$ and $\vec{r}_\omega$ have the same eigenvalue and hence their sum and difference are also the eigenvector. Then from them we obtain two real vectors, instead of complex vectors. When $\omega \in \mathbb{Z}$, we see that the real part of $\vec{r}_\omega$ satisfies the zero boundary condition. That's what we have used in the previous section (for the matrix $A = \text{tridiag}(1, -2, 1)$).

Any discrete initial data can be expanded as a superposition of those eigenfunctions (called modes) and each of them will then be driven by the linear heat equation independent of the others. We would like to determine the dynamics of each mode. Plugging (12.28) into the forward Euler scheme

$$u_j^{n+1} = u_j^n + \frac{\nu k}{h^2}(u_{j+1}^n - 2u_j^n + u_{j-1}^n), \tag{12.29}$$

we get

$$\hat{u}^{n+1}(\omega) = \hat{Q}\hat{u}^n(\Omega), \quad \hat{Q} = 1 + \frac{\nu k}{h^2}\left(2\cos(2\pi\omega h) - 2\right) \tag{12.30}$$

and Q is called the symbol or amplification factor of the discrete operator on the right hand side of (12.29). Note that $\hat{Q} \le 1$ and we want to make sure $\hat{Q} \ge -1$. We then see that $|\hat{Q}| \le 1$ is guaranteed if $\frac{\nu k}{h^2} \le \frac{1}{2}$.

Note that if the initial data is $a_0 e^{2\pi i\omega x}$, the solution is going to be $a(t)e^{2\pi i\omega x}$ with $a(t)$ satisfies

$$\frac{da}{dt} = -\nu(2\pi\omega)^2 a, \qquad a(0) = a_0.$$

So, $a(t) = a_0 e^{-\nu(2\pi\omega)^2 t}$ and the amplification factor for each step is

$$\frac{a((n+1)k)}{a(nk)} = e^{-\nu(2\pi\omega)^2 k}.$$

Note that for the discrete amplification factor $\hat{Q}$,

$$\hat{Q} = 1 + \frac{\nu k}{h^2}\left(2\cos(2\pi\omega h) - 2\right) = 1 - \frac{4\nu k}{h^2}\sin^2(\pi\omega h)$$

$$\hat{Q} - \frac{a((n+1)k)}{a(nk)} = O(k^2) + O(kh^2).$$

This *indicate* (note that so far what we have shown is only a necessary condition for the stability and accuracy) that the overall error should be

$$\begin{aligned}
&\hat{Q}^{T/k} - \left(\frac{a((n+1)k)}{a(nk)}\right)^{T/k} \\
=&\left(\frac{a((n+1)k)}{a(nk)} + O(k^2) + O(kh^2)\right)^{T/k} - \left(\frac{a((n+1)k)}{a(nk)}\right)^{T/k} \\
=&O(k) + O(h^2).
\end{aligned}$$

If we consider the Crank-Nicolson scheme

$$u_j^{n+1} - \frac{\nu k}{2h^2}(u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1}) = u_j^n + \frac{\nu k}{2h^2}(u_{j+1}^n - 2u_j^n + u_{j-1}^n), \quad (12.31)$$

plugging (12.28) into the above, we get

$$\hat{u}^{n+1}(\omega) = \hat{Q}\hat{u}^n(\Omega), \quad \hat{Q} = \frac{1 + \frac{\nu k}{2h^2}\left(2\cos(2\pi\omega h) - 2\right)}{1 - \frac{\nu k}{2h^2}\left(2\cos(2\pi\omega h) - 2\right)}$$

and there is no restriction on k as long as it is nonnegative. One can get

$$\hat{Q}^{T/k} - \left(\frac{a((n+1)k)}{a(nk)}\right)^{T/k} = O(k^2) + O(h^2).$$

12.4 energy method

We first prove stability, then convergence follows from Lax equivalence theorem. The unconditional stability of backward Euler $u_j^{n+1} - u_j^n = \nu k D_+ D_- u_j^{n+1}$ is simple: we multiply both sides of it by u_j^{n+1}, then time h and sum on j, we get

$$h\frac{1}{2}\sum_j |u_j^{n+1}|^2 - |u_j^n|^2 + |u_j^{n+1} - u_j^n|^2 = -\nu k \|D_+ u^{n+1}\|^2,$$

where $\|u^n\|^2 = \sum_{j=1}^{M-1} |u_j^n|^2 h$ and we have used the summation by part $-h\sum_j u_j^n (D_+ D_- u_j^n) = \|D_- u^n\|^2$. So, after summation on n, we get

$$\frac{1}{2}\|u^{N+1}\|^2 + k\nu \sum_{n=1}^{N+1} \|D_+ u^n\|^2 \le \frac{1}{2}\|u^0\|^2.$$

So, backward Euler is unconditionally stable.

To prove the unconditional stability of Crank-Nicolson, multiplying it by $u_j^{n+1} + u_j^n$.

Now let's focus on the forward Euler (12.29)

$$u_j^{n+1} = u_j^n + \nu k D_+ D_- u_j^n.$$

We consider zero boundary condition. The following proof is taken from Gustafsson-Kreiss-Oliger. We square both sides and then multiply h and sum up on j from 1 to $M-1$. We get

$$\|u^{n+1}\|^2 = \|u_j^n\|^2 - 2\nu k \|D_- u^n\|^2 + \nu^2 k^2 \|D_+ D_- u^n\|^2$$

where we have used summation by part to get the 2nd term on the right hand side. Note the following inverse inequality

$$\begin{aligned}\|D_+ y\|^2 &= h^{-2} \sum_{j=1}^{M-1} |y_{j+1} - y_j|^2 h \\ &\le h^{-2} \sum_{j=1}^{M-1} 2(|y_{j+1}|^2 + |y_j|^2) h \le 4h^{-2} \|y\|^2.\end{aligned}$$

So, $\|D_+ D_- u^n\|^2 \le 4h^{-2} \|D_- u^n\|^2$. Therefore

$$\|u^{n+1}\|^2 \le \|u_j^n\|^2 - 2\nu k (1 - \frac{2\nu k}{h^2}) \|D_- u^n\|^2.$$

So, the forward Euler scheme is stable as long as $\frac{2\nu k}{h^2} \le 1$.